Lab 5 : Animation and HUD

Lab Overview:

In Lab 4 you made a character that walks, runs, jumps, falls, rolls and collects coins. In this lab you will turn that prototype into something that feels like a real game, in four stages.

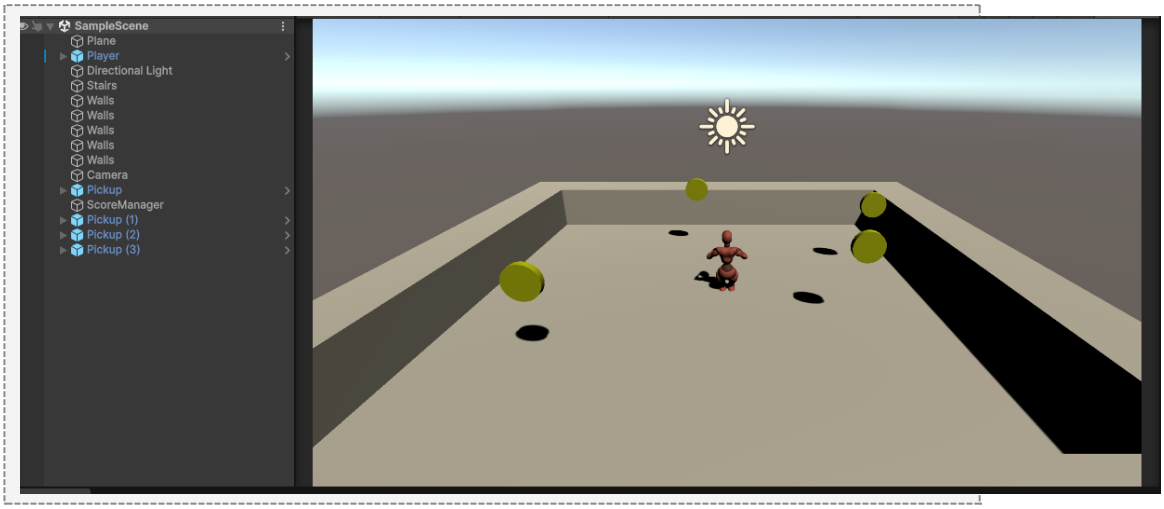
First, you will set up a root-motioned **Flip** animation, bake its movement into the pose, and extend the Animator graph from Lab 4 with a **double jump**: a second jump in mid-air that plays the flip, using a state with several competing transitions. You will then use an **Animation Event** to play a sound at an exact moment of the flip. Next, you will build a **Start Menu** in its own scene and learn how to **switch scenes** from buttons. Then you will add a small **HUD** (heads-up display) that shows the score from Lab 4 and a power-up icon with the time remaining. You will turn the double jump into a **power-up**: a pickup that disappears when collected, unlocks the double jump for 30 seconds, and reappears a few seconds later so it can be collected again. Finally, you will add an **End scene** that the player reaches at the end of the level, which shows their final score.

By the end of the lab you should be able to explain what root motion is and why we sometimes bake it into the pose, how the Animator chooses between several transitions leaving the same state, how an Animation Event lets a clip call your code at an exact frame, why a trigger parameter sometimes needs to be reset, how SceneManager.LoadScene() and the Build Profiles scene list work together, how a static variable carries a value from one scene to the next, how a UI element is anchored to the screen, and why a coroutine is a good fit for "wait, then do something".

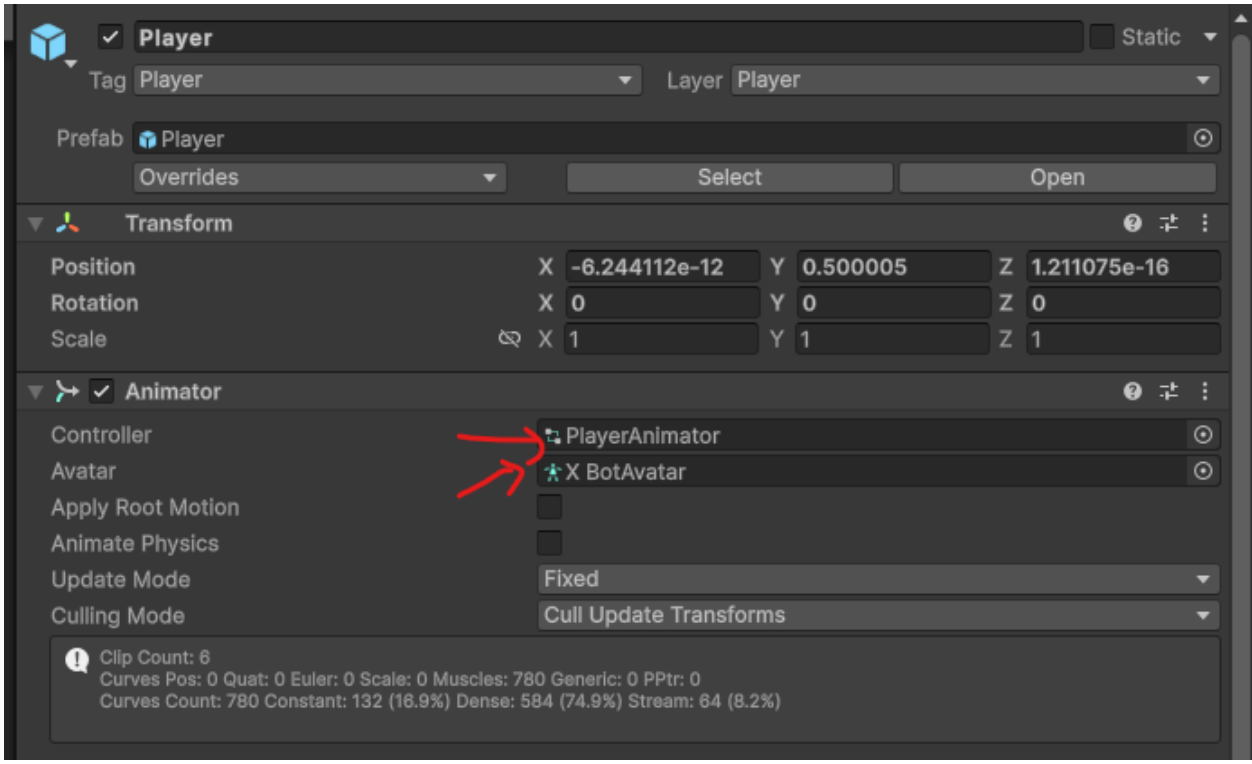
Before You Start:

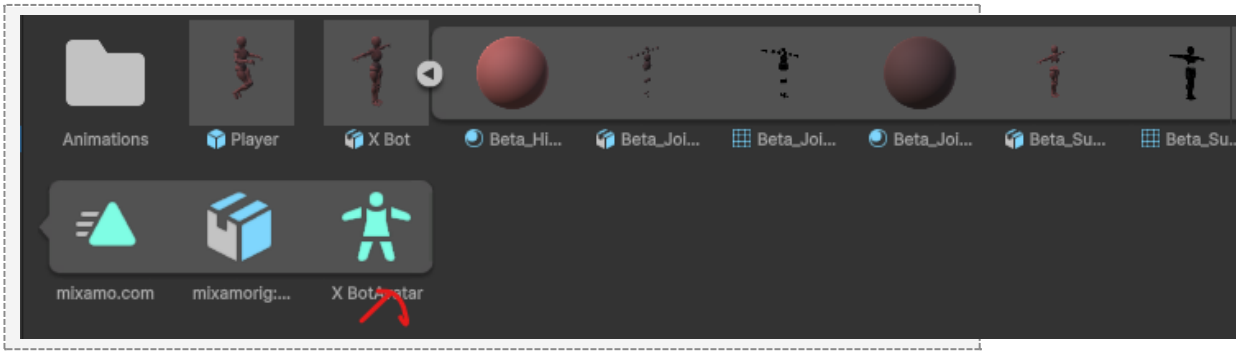
This lab does **not** start from your own Lab 4 project. Everyone starts from the same Lab 5 starter repo, which contains a finished version of Lab 4 plus the new Flip animation.

1. **Get the starter project:**
    - Open the GitHub Classroom assignment (or the repo) and clone it: https://github.com/amcnabbbaltar/450_541_Lab6_2026
    - Open the project in **Unity 6** (the repo was made with **6000.5.7f1**), then open **Assets/Scenes/SampleScene**.



2. **Look at what you are starting with:**
    - **Assets/Scripts**: `PlayerMovement`, `PlayerAnimatorController`, `RollBehaviour`, `ScoreManager` and `ScorePickup`, the same scripts you wrote in Lab 4. Put every new script you create in this folder.
    - **Assets/Character**: the **Player** prefab, the **X Bot** model and an **Animations** folder with **Idle**, **Walking**, **Running**, **Falling**, **StandToRoll** and the new **FrontFlip**, plus the **PlayerAnimator** controller.
    - **PlayerAnimator** has three states, **Idle Walk Run**, **Stand To Roll** and **Falling**, and three parameters, **CharacterSpeed**, **IsGrounded** and **doRoll**.
    - **Assets/Prefab/Pickup**: the coin from Lab 4, with its **CoinSpin** clip in **Assets/CoinAnimation**. A **ScoreManager** object is already in the scene.
    - The **Player** in the scene is tagged **Player** and already has a Rigidbody, a Capsule Collider, **Player Movement** and **Player Animator Controller**.
3. **Check the Player's Avatar:**
    - Select the **Player** in the Hierarchy and look at its **Animator** component.
    - **Avatar** must be **X BotAvatar**. If it shows **None** or **Missing**, click the circle next to the field and choose **X BotAvatar**.
    - Press **Play**: the character should walk, run, jump, fall and roll, and coins should print the score in the Console. If the character slides around frozen in one pose, the Avatar is not set.





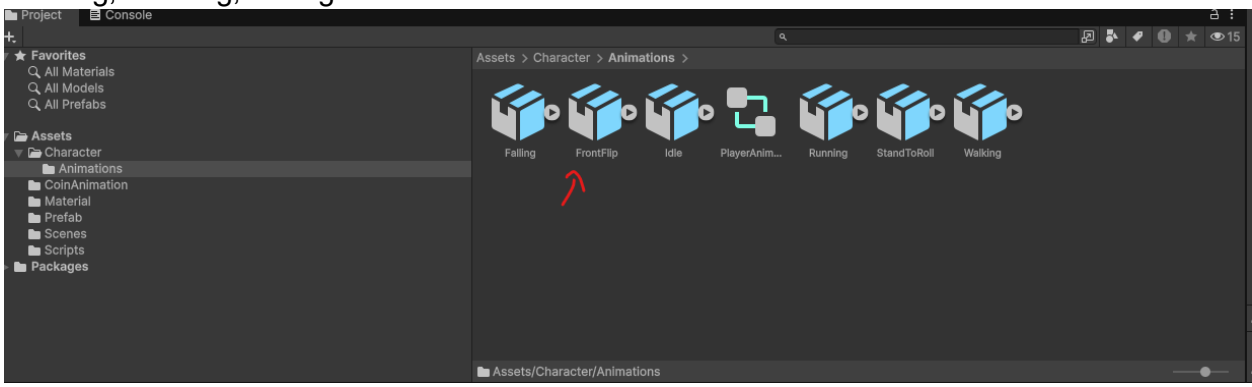
> **Good to know:** The Player in the scene is an **instance of a prefab**. Components and values you add to it in this lab show up in **bold** with a blue line in the Inspector. These are called **overrides** and they are fine.

- Like Lab 4, this lab uses the classic Input Manager (`Input.GetAxisRaw`, `Input.GetButtonDown`).

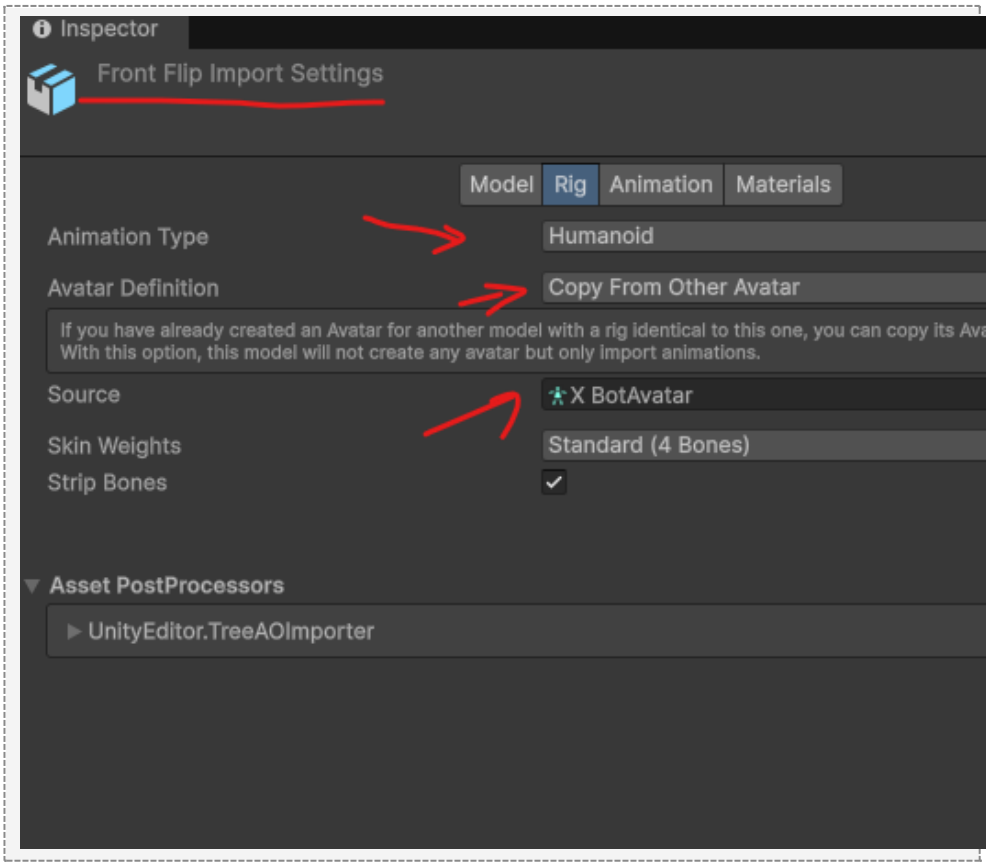
Step-by-Step Solution:Task 1: Configure the Flip Animation and Bake Its Root Motion

This lab adds a **double jump** that plays a **Flip** animation. The clip, **Flip**, is already in the repo, but it is not ready to use: it still has its Mixamo name, and it was made with **root motion**, which would fight with our PlayerMovement script. In this task you will set the clip up and **bake** its movement into the animation.

1. **Find the Flip clip:**
    - Open **Assets/Character/Animations**. Select the **FrontFlip** FBX file, next to Idle, Walking, Running, Falling and StandToRoll.

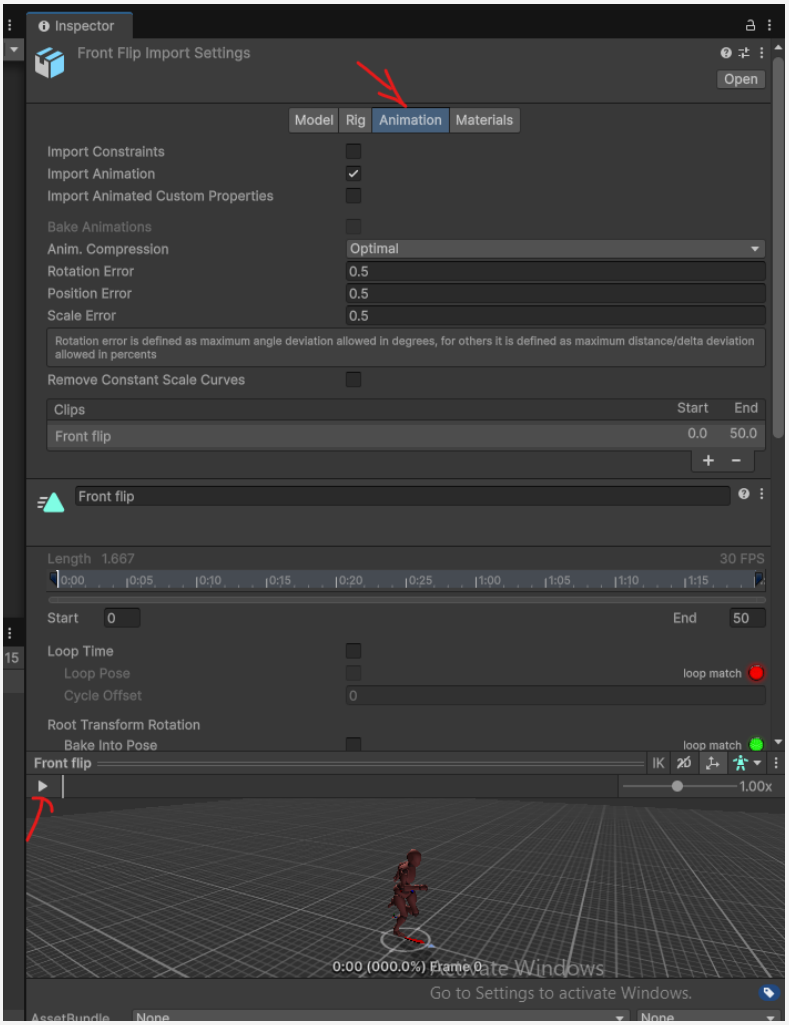


2. **Check the Rig: Humanoid, using the character's Avatar:**
    - With **FrontFlip** selected, open the **Rig** tab in the **Inspector**.
    - **Animation Type** should already be **Humanoid** and **Avatar Definition** should be **Copy From Other Avatar**.
    - Look at **Source**. It must be **X BotAvatar**, the same Avatar the Player uses. If it shows **None** or **Missing**, click the circle and choose **X BotAvatar**.
    - Click **Apply** if you changed anything.



> **Why copy the Avatar?** An Avatar is Unity's map of a humanoid skeleton (where the hips, spine, arms and legs are). When every clip uses the same Avatar, Unity can play any of them on our character without the bones twisting.

3. **See the root motion for yourself:**
    - Press the play button in the preview at the bottom of the Inspector and watch the character's position on the grid.



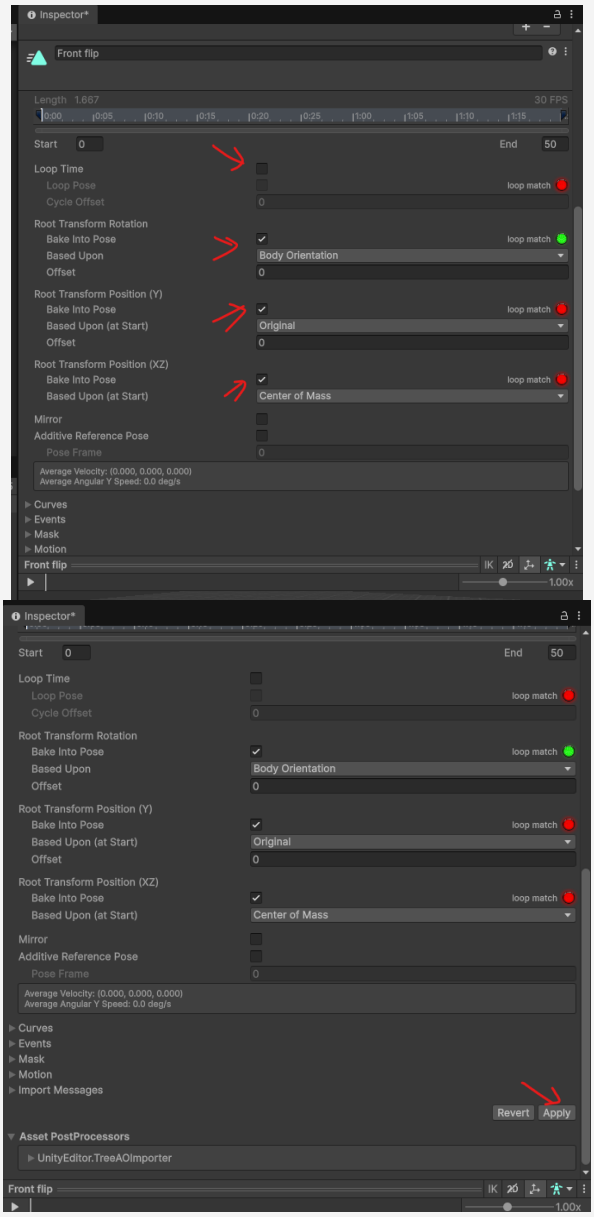
> **What is root motion?** Every humanoid clip has a **root**: one point that stands for "where the character is". In a **root-motioned** clip, the animator moved that root, so the animation itself carries the character through the world. That is great when the animation should drive the movement (our roll in Lab 4 does this, through **RollBehaviour**). It is a problem when a **script** drives the movement: the Rigidbody and the clip would both be trying to move the player.

> **Why is it a problem for the double jump?** PlayerMovement already decides how high and how far the double jump goes. If the Flip clip also moved the player, the character would drift away from its capsule collider, jump higher than the physics says, or end up facing the wrong way after the spin.

4. **Bake the root motion into the pose:**

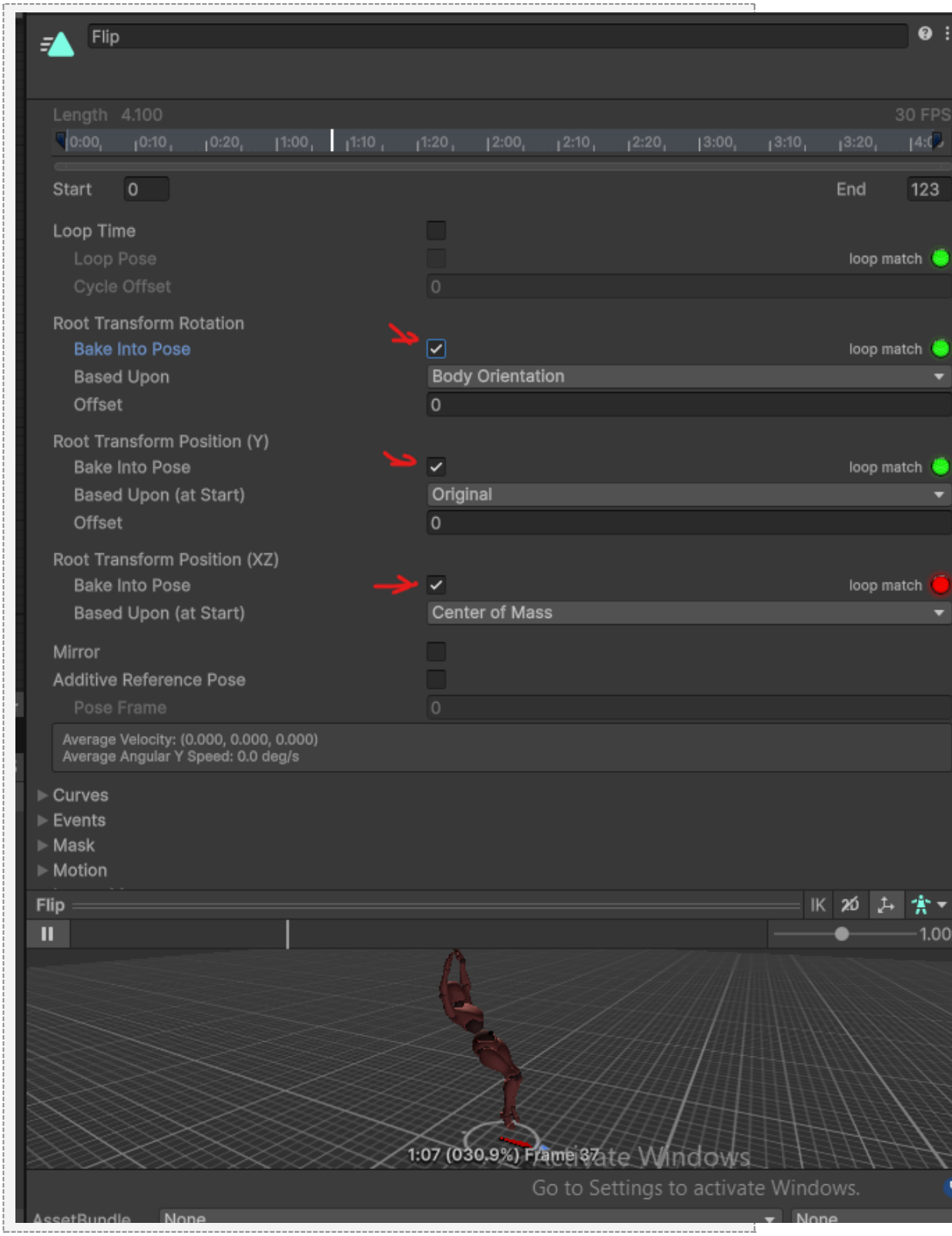
   Still in the **Animation** tab, scroll down to the three **Root Transform** sections. For each one, **Bake Into Pose** tells Unity: "don't move the GameObject with this part of the motion, keep it inside the animated body instead."

   - Make sure **Loop Time** is **unchecked**. A flip should play once, not forever.
   - **Root Transform Rotation**: check **Bake Into Pose**, **Based Upon: Original**. The spin stays in the body, so the Player object never turns.
   - **Root Transform Position (Y)**: check **Bake Into Pose**, **Based Upon: Original**. The up-and-down movement stays in the body, so the height of the jump comes only from physics.
   - **Root Transform Position (XZ)**: check **Bake Into Pose**, **Based Upon: Original**. The forward travel stays in the body, so the horizontal movement comes only from PlayerMovement.
   - Look at the small coloured circles next to each section (**loop match**). They are not important for a clip that doesn't loop.
   - Click **Apply**.



> **What does Bake Into Pose do?** It moves that part of the root motion **out of the root and into the pose**. The body still spins and tucks on screen, but the root, and so the Player GameObject, stays exactly where our script puts it. With all three boxes checked, the clip has **no root motion left**.

5. **Check the result:**
    - Play the preview again. The character still flips, but the root no longer travels across the grid.
    - In the Animator component on the Player, **Apply Root Motion** should be **unchecked**. (RollBehaviour only switches it on during the roll.)
    - You will see the real result in Task 4: the character's body should stay inside its green capsule collider for the whole flip.


Flip
Length 4.100
30 FPS
0:00
0:10
0:20
1:00
1:10
1:20
2:00
2:10
2:20
3:00
3:10
3:20
Start 0
End 123
Loop Time
Loop Pose
loop match
Cycle Offset 0
Root Transform Rotation
Bake Into Pose
loop match
Based Upon Body Orientation
Offset 0
Root Transform Position (Y)
Bake Into Pose
loop match
Based Upon (at Start) Original
Offset 0
Root Transform Position (XZ)
Bake Into Pose
loop match
Based Upon (at Start) Center of Mass
Mirror
Additive Reference Pose
Pose Frame 0
Average Velocity: (0.000, 0.000, 0.000)
Average Angular Y Speed: 0.0 deg/s
Curves
Events
Mask
Motion
Flip
IK
1.00
1:07 (030.9%) Frame 37
Activate Windows
Go to Settings to activate Windows.
AssetBundle
None
None

6. **Check your understanding:**
    - In your own words, what is the difference between a root-motioned animation and an "in place" animation?

Root-motioned animation will move the Root as the animation plays (e.g. running), while an in place animation will only move the rig, but not its Root, which is ideal for an animation like Flip.

Task 2: Add the Flip State to the Animator

In Lab 4 the Base Layer had three states: **Idle Walk Run**, **Stand To Roll** and **Falling**. You will now add a fourth state, **Flip**, directly in the Base Layer. It is different from the others in one important way: it can only be reached **from Falling**, so the Animator itself guarantees the flip never plays on the ground.

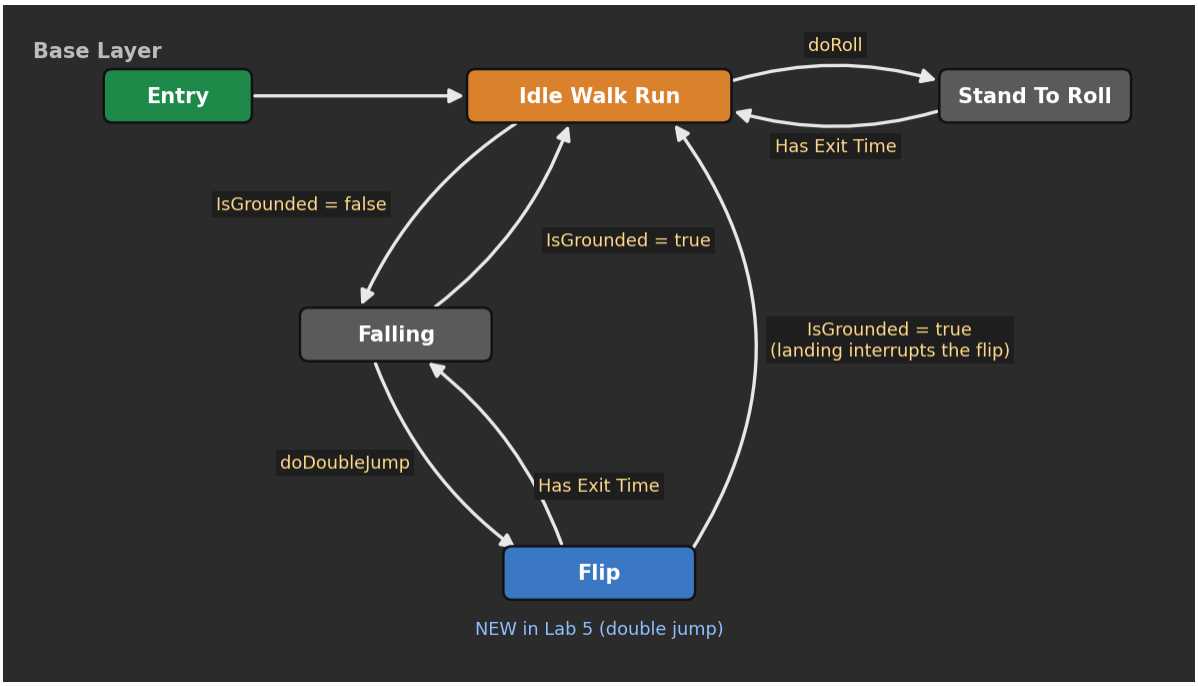
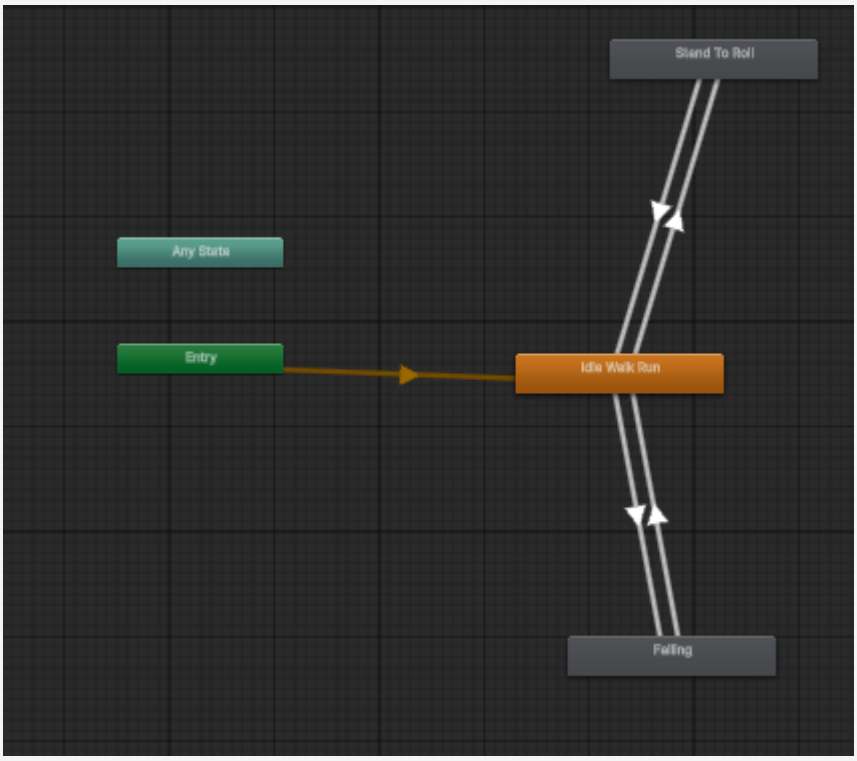
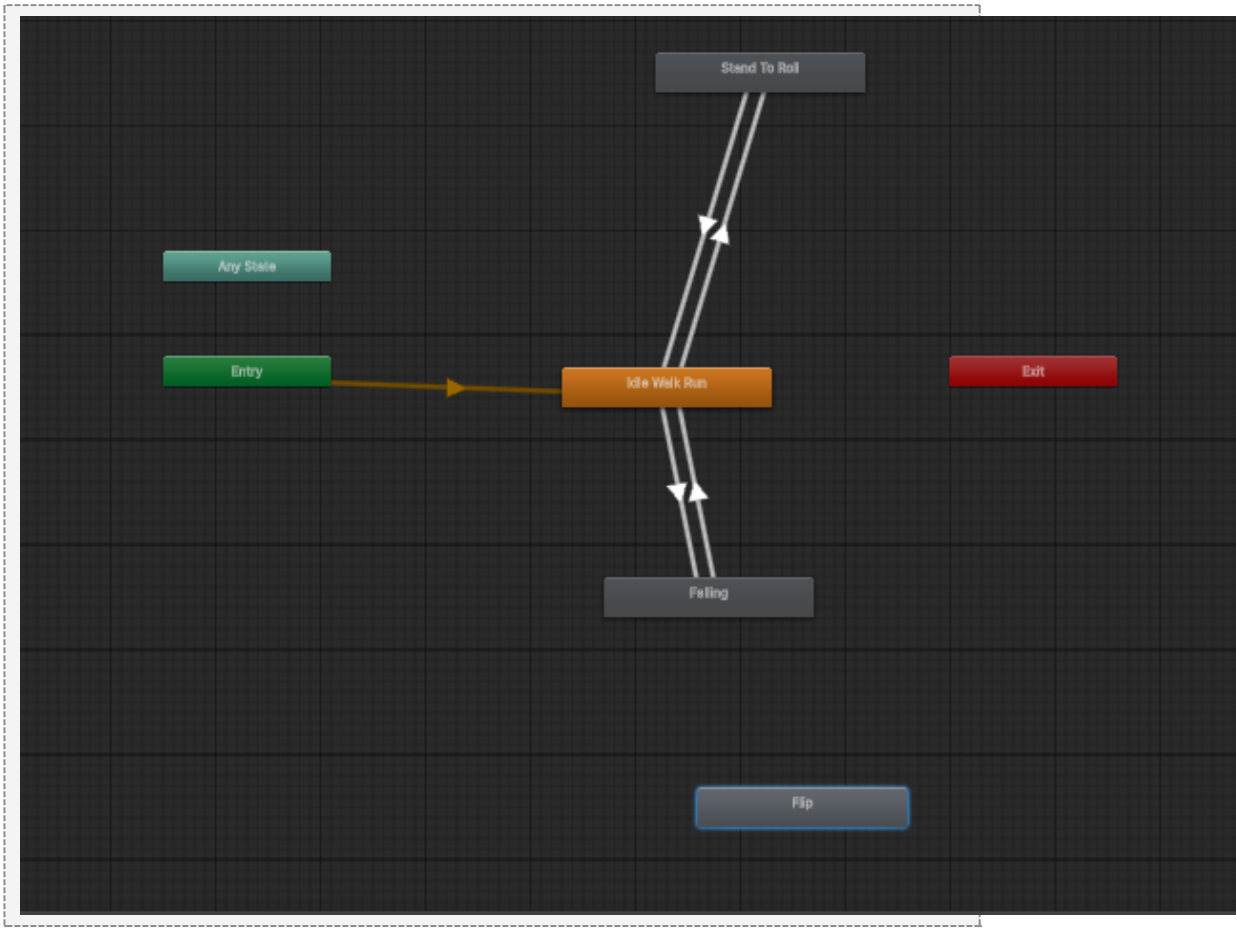


Figure 1: The Base Layer you will have at the end of Task 3

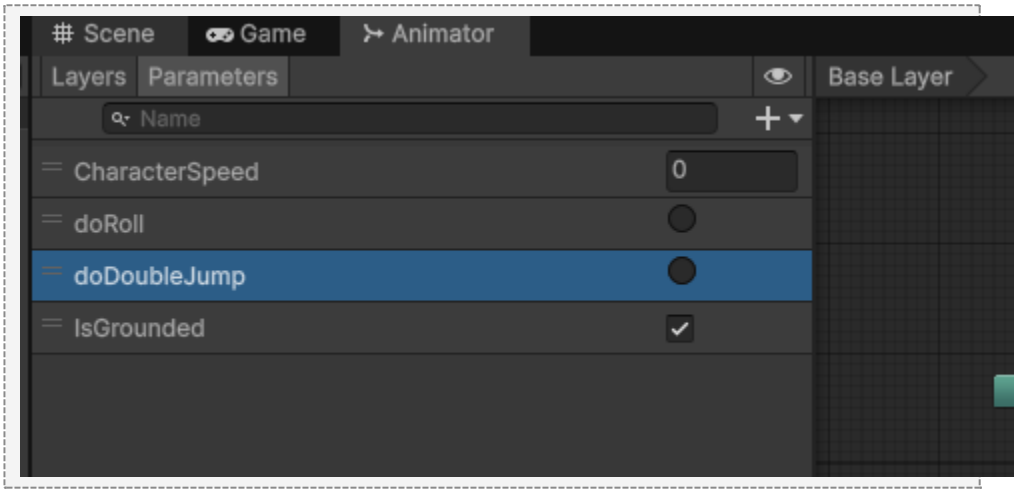
1. **Open the Animator and check the graph:**
    - Open the **PlayerAnimator** controller (double-click it in **Assets/Character/Animations**).
    - You should see **Idle Walk Run** (orange), **Stand To Roll** and **Falling**, connected exactly as in Lab 4. **Do not delete or change any of them.**
    - Check the two Falling transitions still have their conditions: **IsGrounded = false** going into Falling, and **IsGrounded = true** coming back.



2. **Add the Flip state:**
    - Expand the **FrontFlip** FBX in the Project window (click the small arrow on the file) and drag its **Flip** clip into an empty area of the Animator, below **Falling**.
    - A new grey state called **Flip** appears. It has no arrows yet, so the Animator can never reach it.



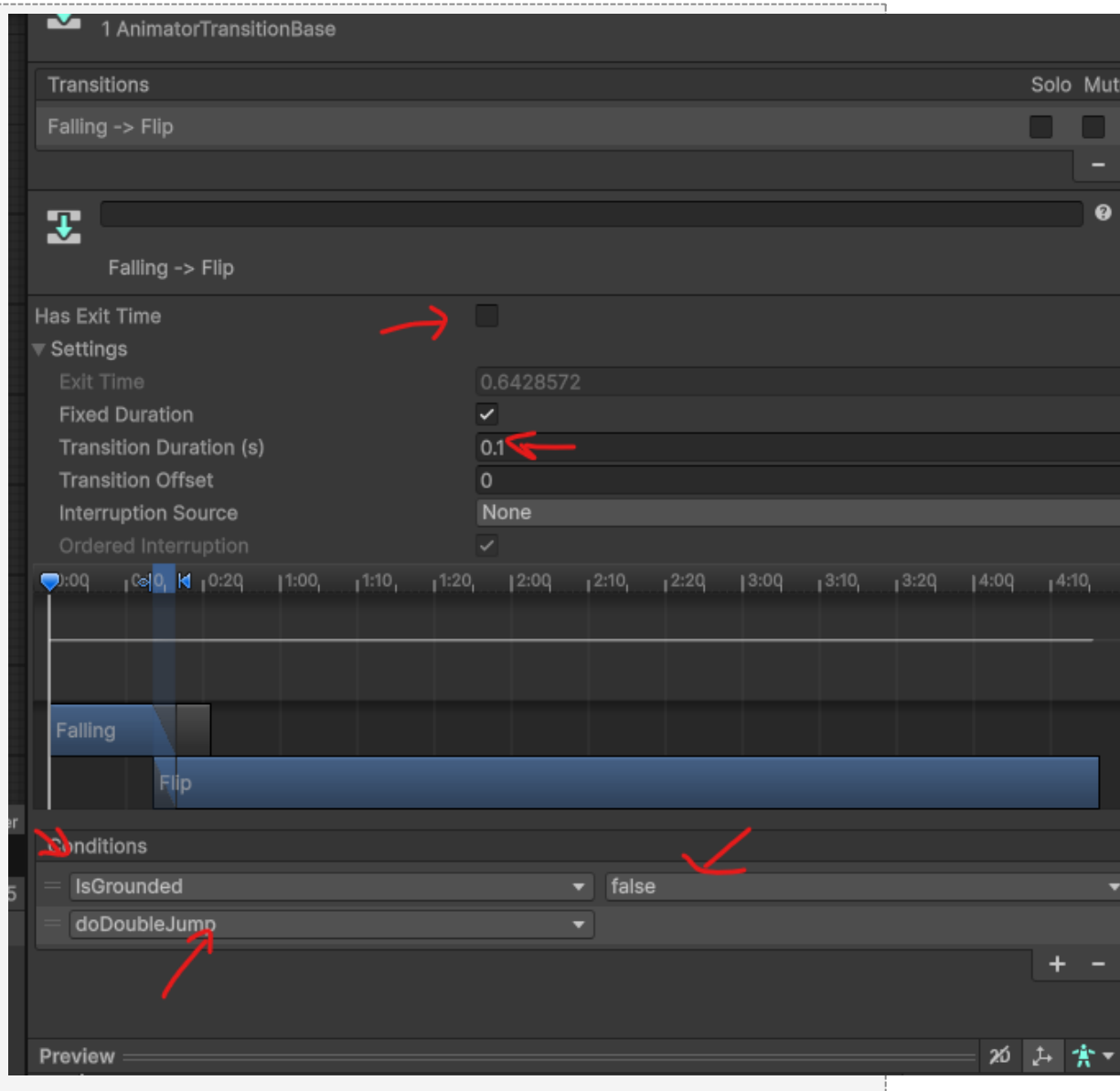
3. **Add the doDoubleJump parameter:**
    - In the **Parameters** tab, click **+** and add a **Trigger** called `doDoubleJump`.
    - You should now have four parameters: CharacterSpeed (Float), IsGrounded (Bool), doRoll (Trigger) and doDoubleJump (Trigger).

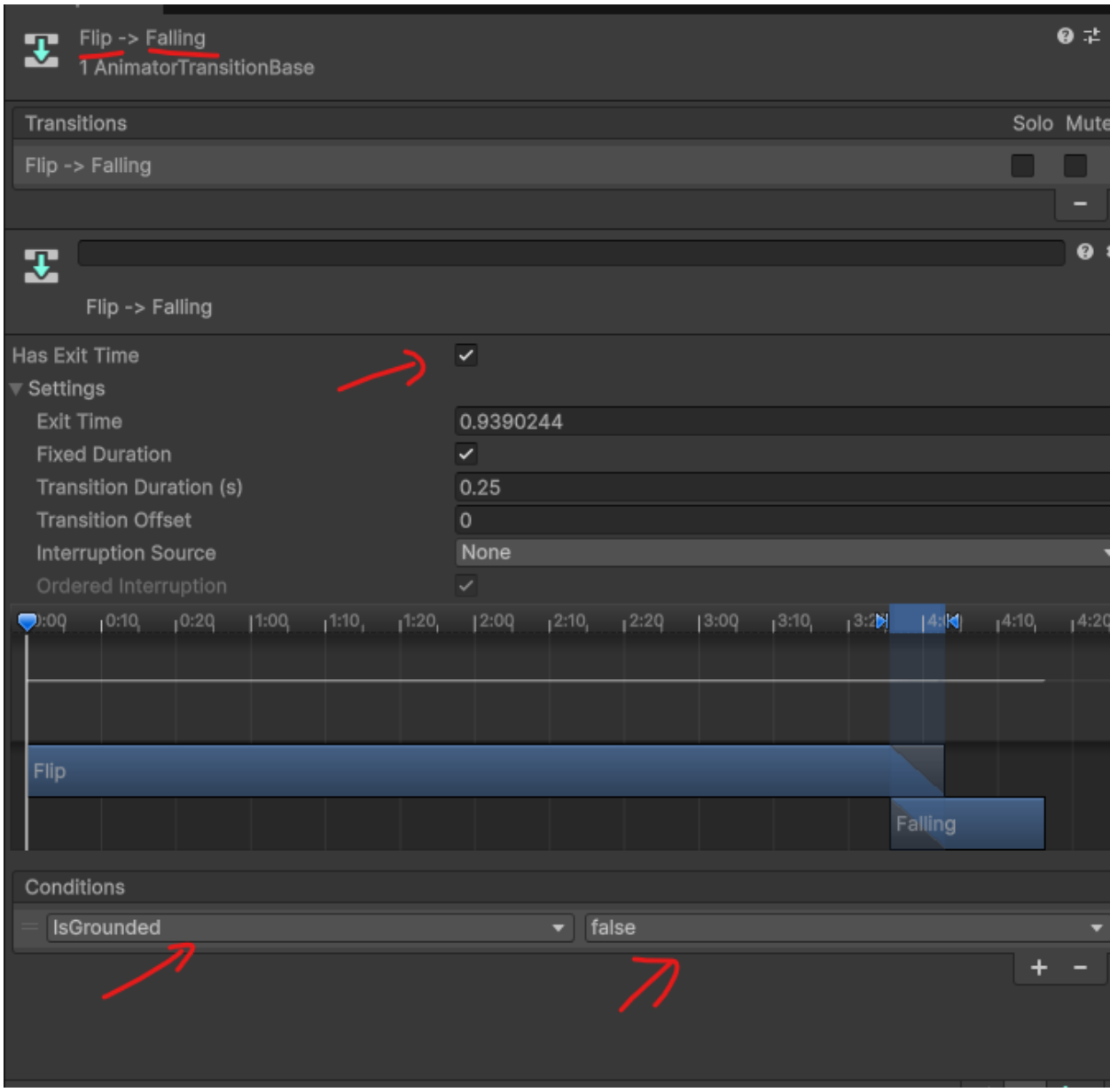


Task 3: Create the Flip Transitions

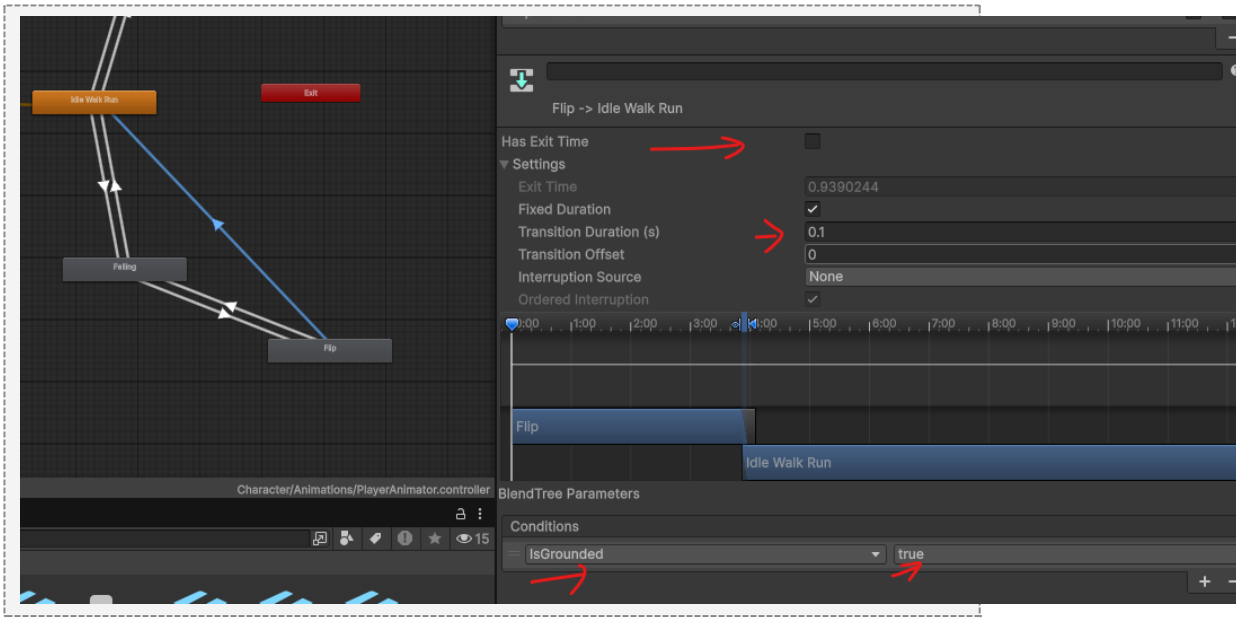
The rule we want is simple to say: **the flip can only start while the player is in the air, and landing always wins.** Flip will have one transition coming in and **two** going out.

1. **Falling → Flip (start the double jump):**
    - Right-click **Falling** > **Make Transition** > click **Flip**.
    - Condition: **doDoubleJump**. Uncheck **Has Exit Time** so the flip starts the moment the button is pressed. Transition Duration: 0.1.
    - Because this is the **only** arrow into Flip, the character must already be in Falling (in the air) for the flip to play.



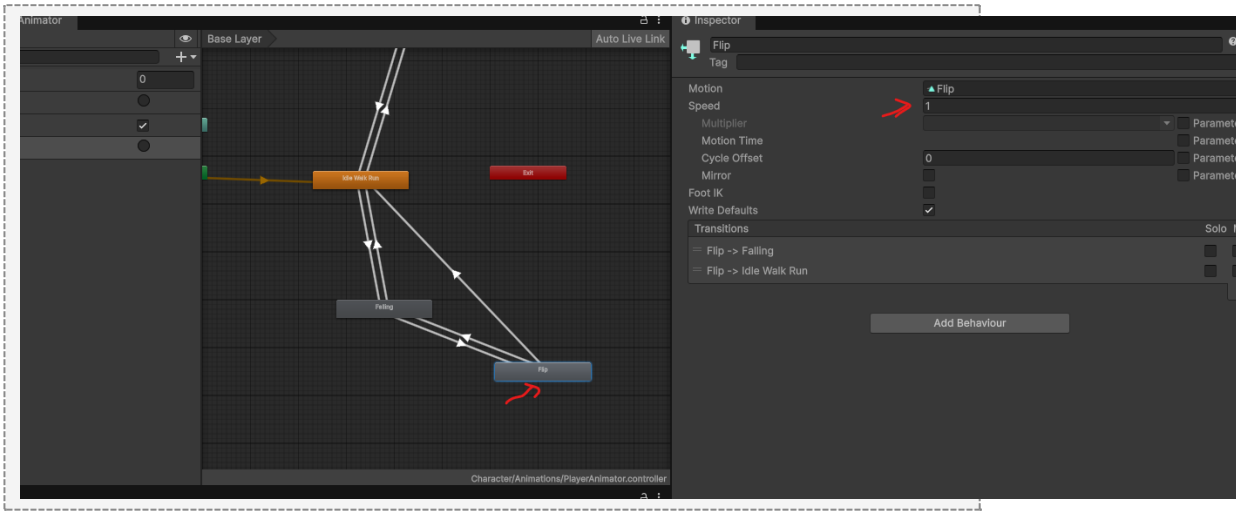
2. **Flip → Falling (flip finished, still in the air):**
    - Right-click **Flip** > **Make Transition** > click **Falling**.
    - **No condition.** Leave **Has Exit Time** checked and set **Exit Time** to 0.9 and **Transition Duration** to 0.15.
    - This is the same idea as the roll in Lab 4: the flip plays (almost) to the end before returning to Falling.
    - 

3. **Flip → Idle Walk Run (landing interrupts the flip):**
    - Right-click **Flip** > **Make Transition** > click **Idle Walk Run**.
    - Condition: **IsGrounded = true**. Uncheck **Has Exit Time**. Transition Duration: 0.1.



> **Why two ways out of Flip?** If the player is still in the air when the flip ends, **Has Exit Time** sends it back to Falling. If the player lands in the middle of the flip, the **IsGrounded = true** transition fires right away, because it doesn't wait for an exit time. Without it, the character would finish flipping while standing on the ground.
>
> **Which transition wins?** When a state has several outgoing transitions, Unity checks them from top to bottom in the **Transitions** list of the state's Inspector and takes the first one whose conditions are true. Select **Flip** and drag the **IsGrounded = true** transition to the top of that list so landing always has priority.

4. **Set the playback speed:**
    - Select the **Flip** state. In the Inspector, set **Speed** to **2**.
    - The clip is 51 frames long (about 1.7 seconds), but a double jump only lasts about a second, so the flip needs to play faster. **Speed** multiplies the clip's playback rate without changing the clip file itself. You will tune this value once you can test.


Animator
Base Layer
Auto Live Link
0
Idle Walk Run
Exit
Falling
Flip
Character/Animations/PlayerAnimator.controller
Inspector
Flip
Tag
Motion
Flip
Speed
1
Multiplier
Motion Time
Cycle Offset
0
Mirror
Foot IK
Write Defaults
Transitions
Solo
Flip -> Falling
Flip -> Idle Walk Run
Add Behaviour

Task 4: Add the Double Jump and Trigger the Flip from Code

The Animator is ready, but nothing sends `doDoubleJump` yet, and the player can't actually jump twice. In this task you will add a real **double jump** to PlayerMovement, then update PlayerAnimatorController so the flip plays with it. For now the double jump is always available; in Task 9 you will lock it behind a power-up.

1. **Add a double jump to PlayerMovement:**
   The jump code lives in **PlayerMovement**, so that is where the double jump belongs. Open **PlayerMovement** (in **Assets/Scripts**) and make these five changes.
   - **1.** Under the **Movement Settings** header, add a force for the second jump:

```
[SerializeField] private float doubleJumpForce = 4f;
```

   - **2.** Next to `private bool jumpRequested = false;`, add two more flags:

```
private bool doubleJumpRequested = false; // set in TryDoubleJump()
private bool hasDoubleJumped = false;     // one per jump
```

   - **3.** In **Update()**, just below the ground check line, give the double jump back when the player lands:

```
// Back on the ground: the double jump is available again
if (isGrounded)
{
    hasDoubleJumped = false;
}
```

   - **4.** Replace **FixedUpdate()** and **Jump()** with these versions. `Jump` now receives the force to use, so the same method works for both jumps:

```
private void FixedUpdate()
{
    MovePlayer();

    if (jumpRequested)
    {
        Jump(jumpForce);
        jumpRequested = false;
    }

    // NEW: the second jump, in mid-air
    if (doubleJumpRequested)
    {
        Jump(doubleJumpForce);
        doubleJumpRequested = false;
    }
}

private void Jump(float force)
{
    rb.linearVelocity = new Vector3(rb.linearVelocity.x, 0f,
                                    rb.linearVelocity.z);
    rb.AddForce(Vector3.up * force, ForceMode.VelocityChange);
```

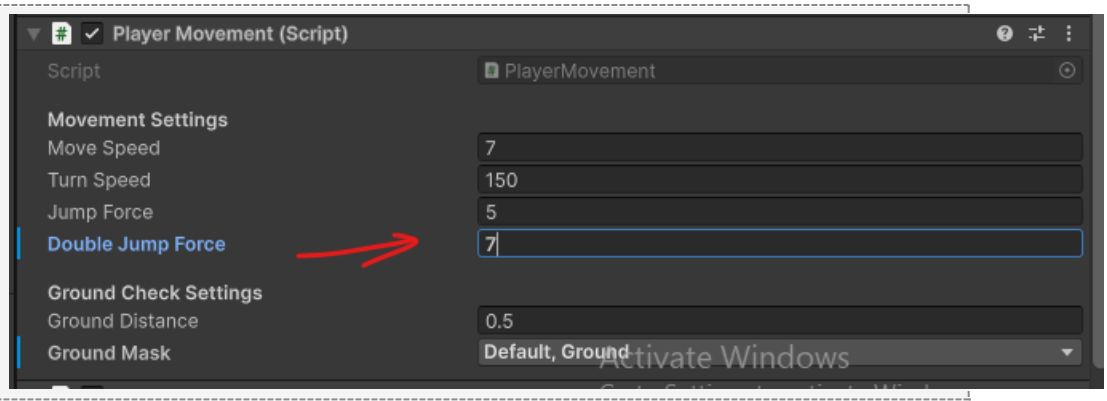
```
}
```

- **5.** Add this public method at the bottom of the class:

```
// Tries to jump a second time in mid-air.
// Returns true if the double jump really happened.
public bool TryDoubleJump()
{
    // Not allowed on the ground, or if it was already used
    if (isGrounded || hasDoubleJumped) return false;

    doubleJumpRequested = true;
    hasDoubleJumped = true;
    return true;
}
```

- `TryDoubleJump()` only sets a flag, exactly like the Space key does with `jumpRequested`. The physics still happens in **FixedUpdate()**, where Rigidbody changes belong.
- It returns a **bool** so the caller knows whether the jump happened. We will use that to play the Flip animation **only** when the player really jumps.
- `Jump()` zeroes the vertical velocity first (from Lab 4), so the double jump gains the same height whether the player is still rising or already falling.
- Back in Unity, **Double Jump Force** appears on the Player Movement component. 4 is a little weaker than the normal jump (5): tune it to taste. (higher than 5 will make the double jump higher than the first jump)



2. **Update PlayerAnimatorController to send doDoubleJump:**

   Open **PlayerAnimatorController**. When the player presses **Space** again in mid-air, it asks PlayerMovement for a double jump and, if that worked, starts the flip. Replace the whole script with:

```
using UnityEngine;

// Sends the player's movement to the Animator every frame
// so the right animation plays at the right time.
public class PlayerAnimatorController : MonoBehaviour
{
    private Animator animator;       // plays the animations
```

```
    private PlayerMovement movement; // our movement script
    private Rigidbody rb;            // to read the player's speed

    private void Start()
    {
        animator = GetComponent<Animator>();
        movement = GetComponent<PlayerMovement>();
        rb = GetComponent<Rigidbody>();
    }

    private void Update()
    {
        // Send the current speed to the blend tree
        animator.SetFloat("CharacterSpeed", rb.linearVelocity.magnitude);

        // On the ground? (controls Falling and ends the flip)
        bool grounded = movement.IsGrounded;
        animator.SetBool("IsGrounded", grounded);

        // When Fire1 (left mouse button / left Ctrl) is released, roll
        if (Input.GetButtonUp("Fire1"))
        {
            animator.SetTrigger("doRoll");
        }

        if (grounded)
        {
            // NEW: throw away a double jump request that was never used
            animator.ResetTrigger("doDoubleJump");
        }
        else
        {
            // NEW: Double jump. Space again in mid-air.
            // The flip only plays if PlayerMovement really jumped.
            if (Input.GetButtonDown("Jump") && movement.TryDoubleJump())
            {
                animator.SetTrigger("doDoubleJump");
            }
        }
    }
}
```

- `Input.GetButtonDown("Jump") && movement.TryDoubleJump()`: C# only evaluates the right side if the left side is true, so `TryDoubleJump()` is only called on the frame Space is pressed.
- The jump and the animation can never get out of sync: no jump, no flip.

> **Why ResetTrigger?** A trigger stays "on" until a transition uses it. Imagine the player double jumps while the Animator is in a state with no arrow to Flip, for example after rolling off a ledge. The trigger is never used, so it waits, and the character would flip all by itself the next time it falls. Calling `ResetTrigger("doDoubleJump")` on the ground cancels any leftover request.

3. **Test the double jump:**
    - Press **Play**. Jump with **Space**, then press **Space again** in the air: the player jumps a second time and flips, then falls and lands into Idle Walk Run.
    - Press Space a third time in the air: nothing happens. The double jump only works **once** per jump.
    - Try landing on a higher platform in the middle of the flip: the flip should stop cleanly.
    - Tune **Speed** on the Flip state and **Double Jump Force** until the flip finishes just before landing.
    - Select the Player while the game runs and watch the Animator window: you will see the blue progress bar move from Idle Walk Run to Falling, to Flip and back.

> **Player jumps twice but doesn't flip?** Check the **Falling → Flip** transition uses **doDoubleJump** with **Has Exit Time** off, and that the parameter name matches the script exactly (`doDoubleJump`, lowercase d).
>
> **Character keeps flipping after landing?** The **Flip → Idle Walk Run** transition is missing, or it still has **Has Exit Time** checked.
>
> **Player ends up facing another way, or the body leaves the capsule during the flip?** The clip still has root motion. Go back to Task 1 and check that **Bake Into Pose** is ticked for **Rotation**, **Position (Y)** and **Position (XZ)**, then click **Apply**.

4. **Check your understanding:**
    - Flip has two outgoing transitions. Which one is taken if the player lands halfway through the flip, and why?

If the Player lands halfway through the flip, the animation will take the Idle Walk Run transaction and stop the Flip animation.

    - There is no transition from Idle Walk Run to Flip. What does that guarantee, even if the script sent `doDoubleJump` by mistake while on the ground?

It guarantees that the Player cannot Flip when grounded.

    - `TryDoubleJump()` only sets `doubleJumpRequested = true` instead of calling `Jump()` right away. Why? (Think back to Update() vs. FixedUpdate() in Lab 4.)

Because we should handle input detections in Update() and apply the physics changes in FixedUpdate().

- Why does `TryDoubleJump()` return a bool instead of void? What would go wrong in PlayerAnimatorController without it?

Because it determines if the double jump happened already. Without a return, the animator would have no way to know if the player is grounded, or if the player has already double jumped, and would trigger a double jump anyway.

Task 6: Build a Start Menu Scene

Up to now, pressing Play drops you straight into the level. Real games start on a menu. A **scene** is a separate level file: our menu will live in its own scene called **StartMenu**, and the level you have been building stays in **SampleScene**.

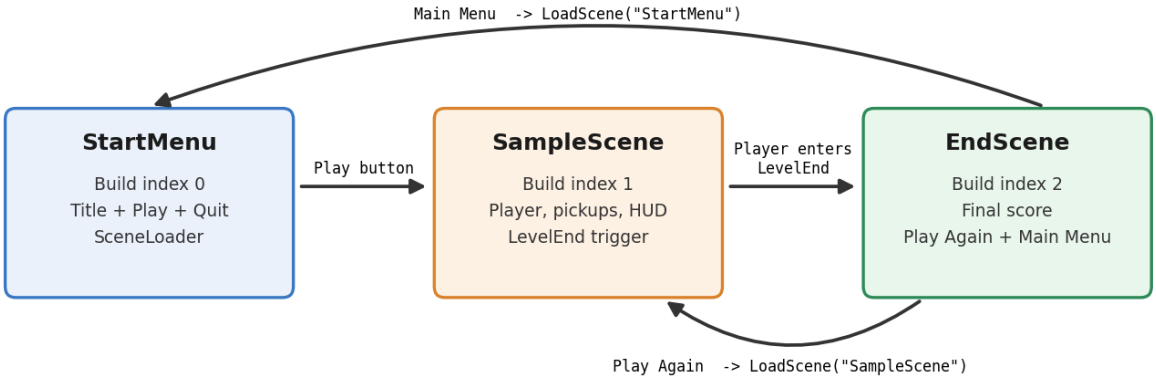
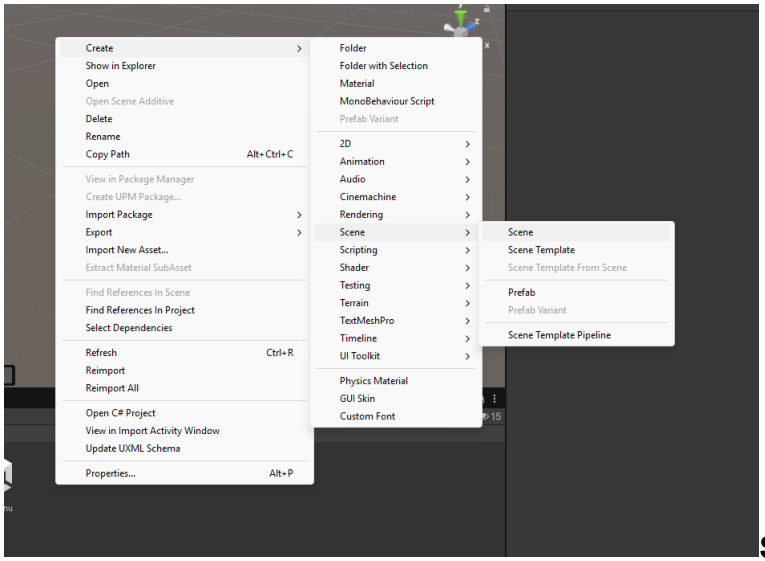
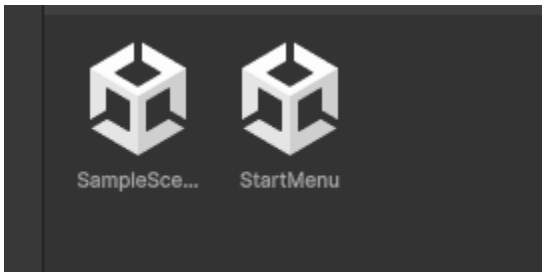


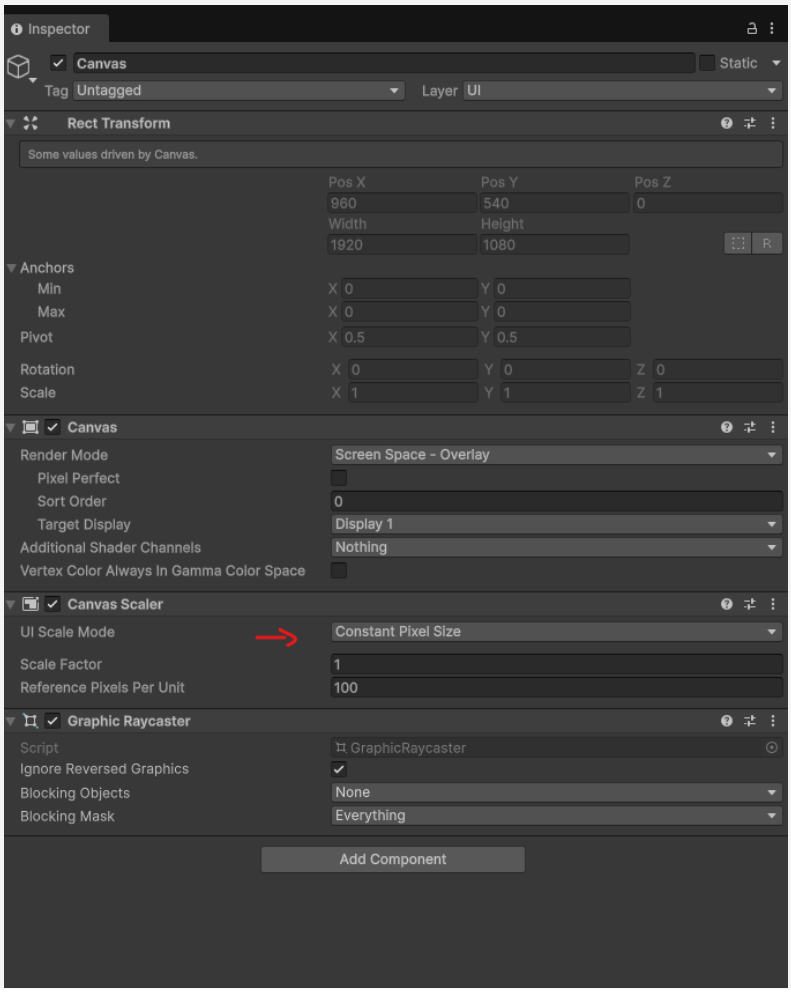
Figure 2: The three scenes you will have by the end of this lab (EndScene is built in Task 10)

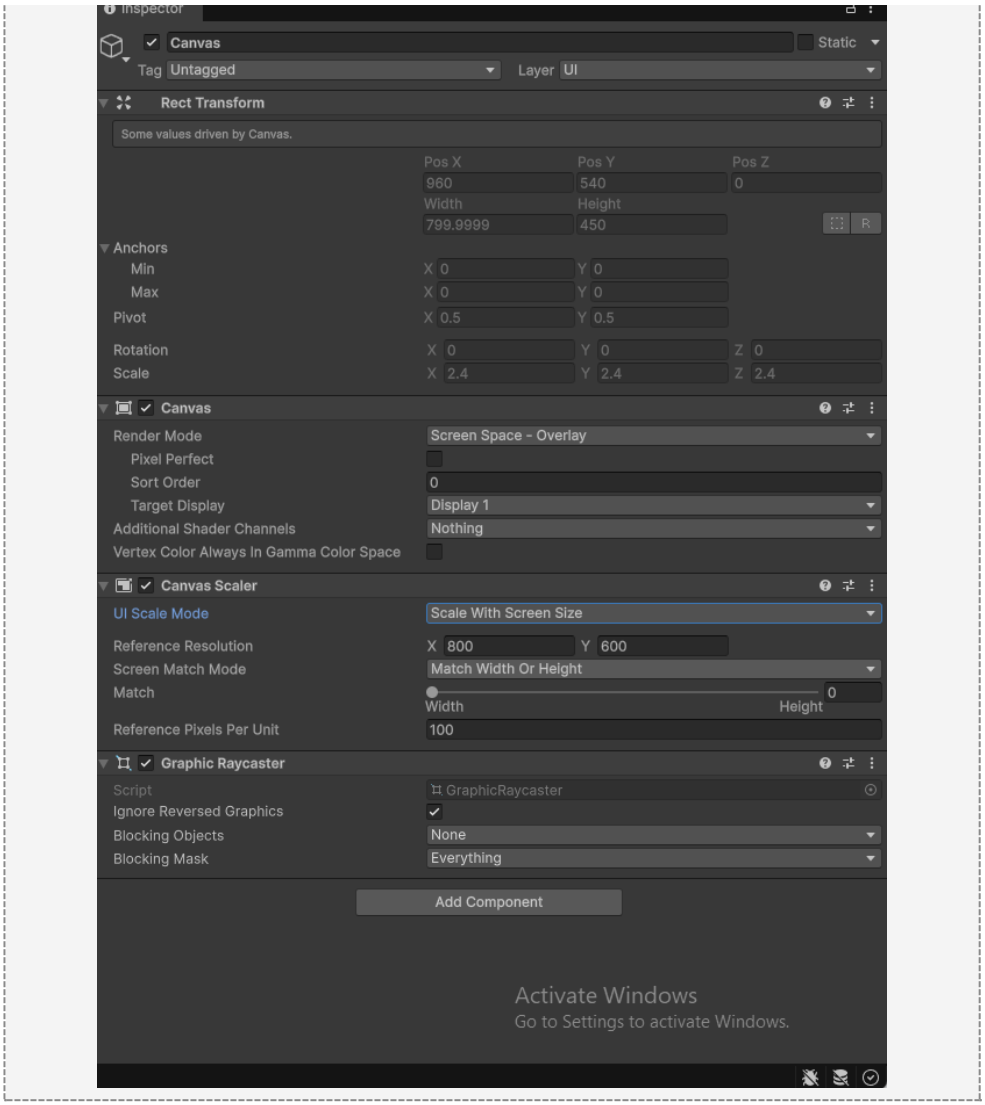
1. **Create the StartMenu scene:**
    - Save SampleScene first (**Ctrl+S**).
    - In the Project window open the **Scenes** folder, right-click and choose **Create > Scene** (in some Unity 6 versions: **Create > Scene > Scene**). Name it

- 

StartMenu.
  - Double-click **StartMenu** to open it. It only contains a Main Camera and a Directional Light.
  - 

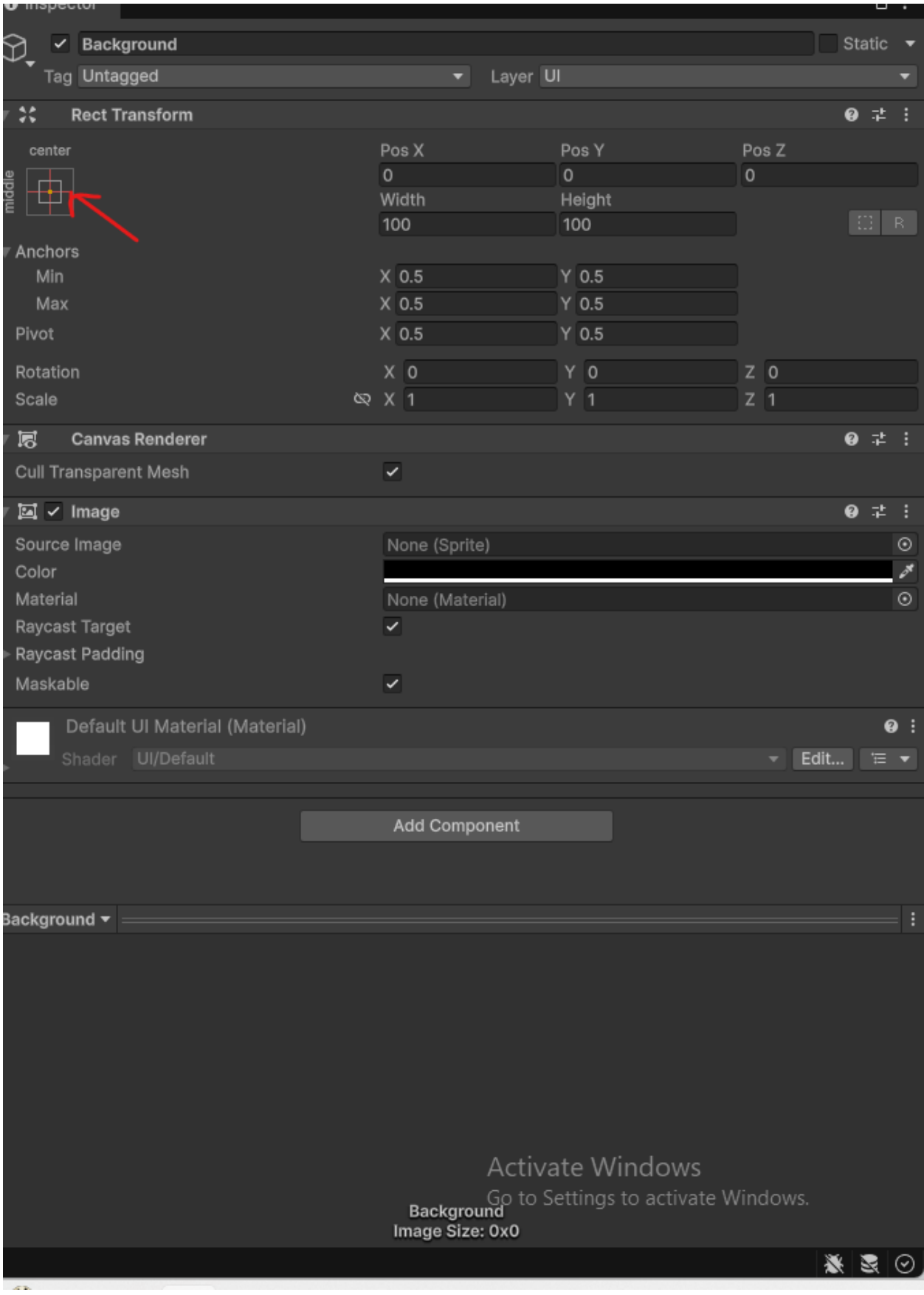
2. **Create a Canvas:**
   - In the Hierarchy right-click > **UI > Canvas**. Unity also creates an **EventSystem**: this is what lets the mouse click buttons, so don't delete it.
   - Select the Canvas. In the **Canvas Scaler** component set **UI Scale Mode** to **Scale With Screen Size**, **Reference Resolution** to **1920 x 1080**, and **Match** to **0.5**.

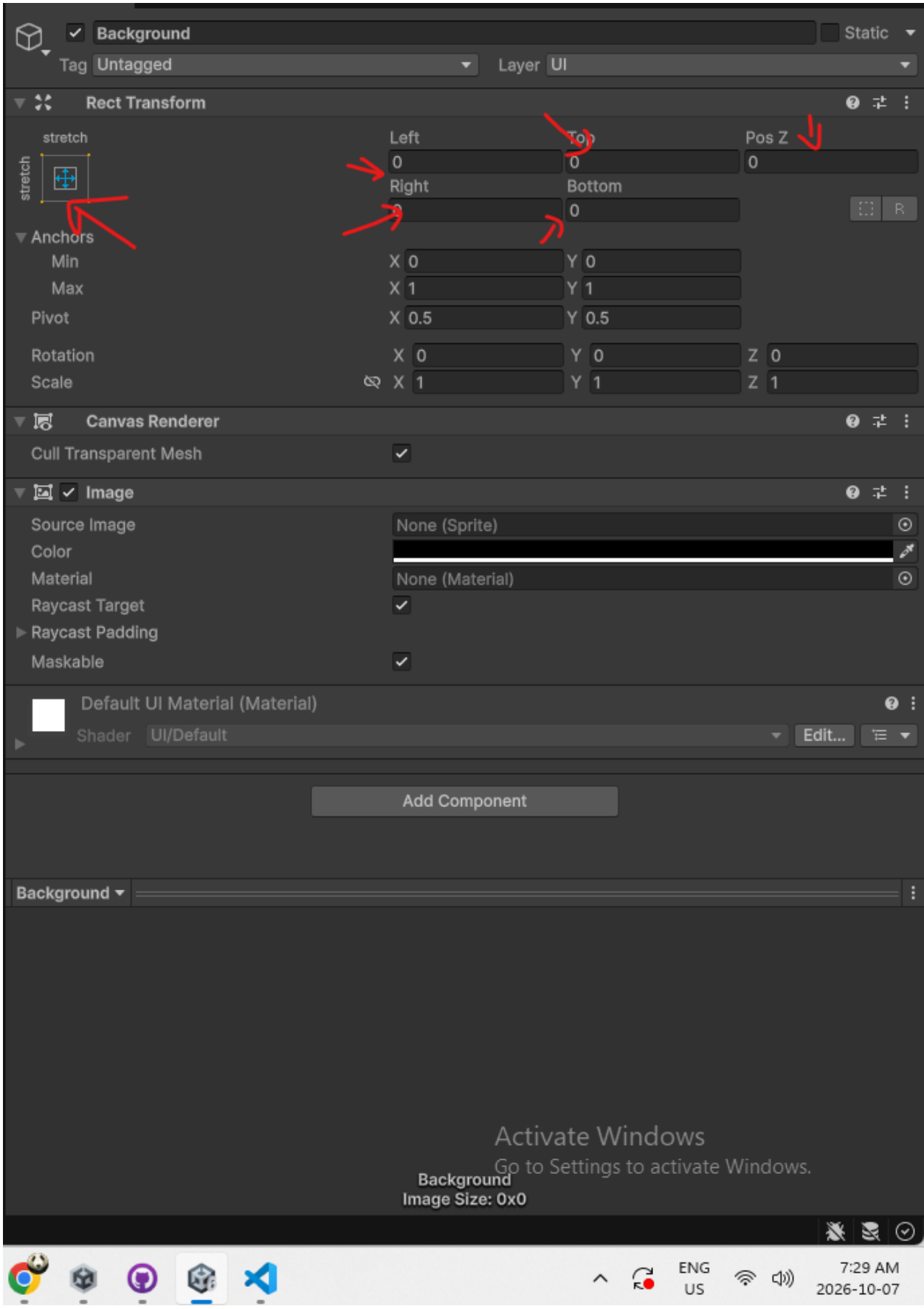

Inspector
Canvas
Static
Tag Untagged
Layer UI
Rect Transform
Some values driven by Canvas.
Pos X
Pos Y
Pos Z
960
540
0
Width
Height
1920
1080
R
Anchors
Min
Max
Pivot
Rotation
Scale
Canvas
Render Mode
Screen Space - Overlay
Pixel Perfect
Sort Order
0
Target Display
Display 1
Additional Shader Channels
Nothing
Vertex Color Always In Gamma Color Space
Canvas Scaler
UI Scale Mode
Constant Pixel Size
Scale Factor
1
Reference Pixels Per Unit
100
Graphic Raycaster
Script
GraphicRaycaster
Ignore Reversed Graphics
Blocking Objects
None
Blocking Mask
Everything
Add Component



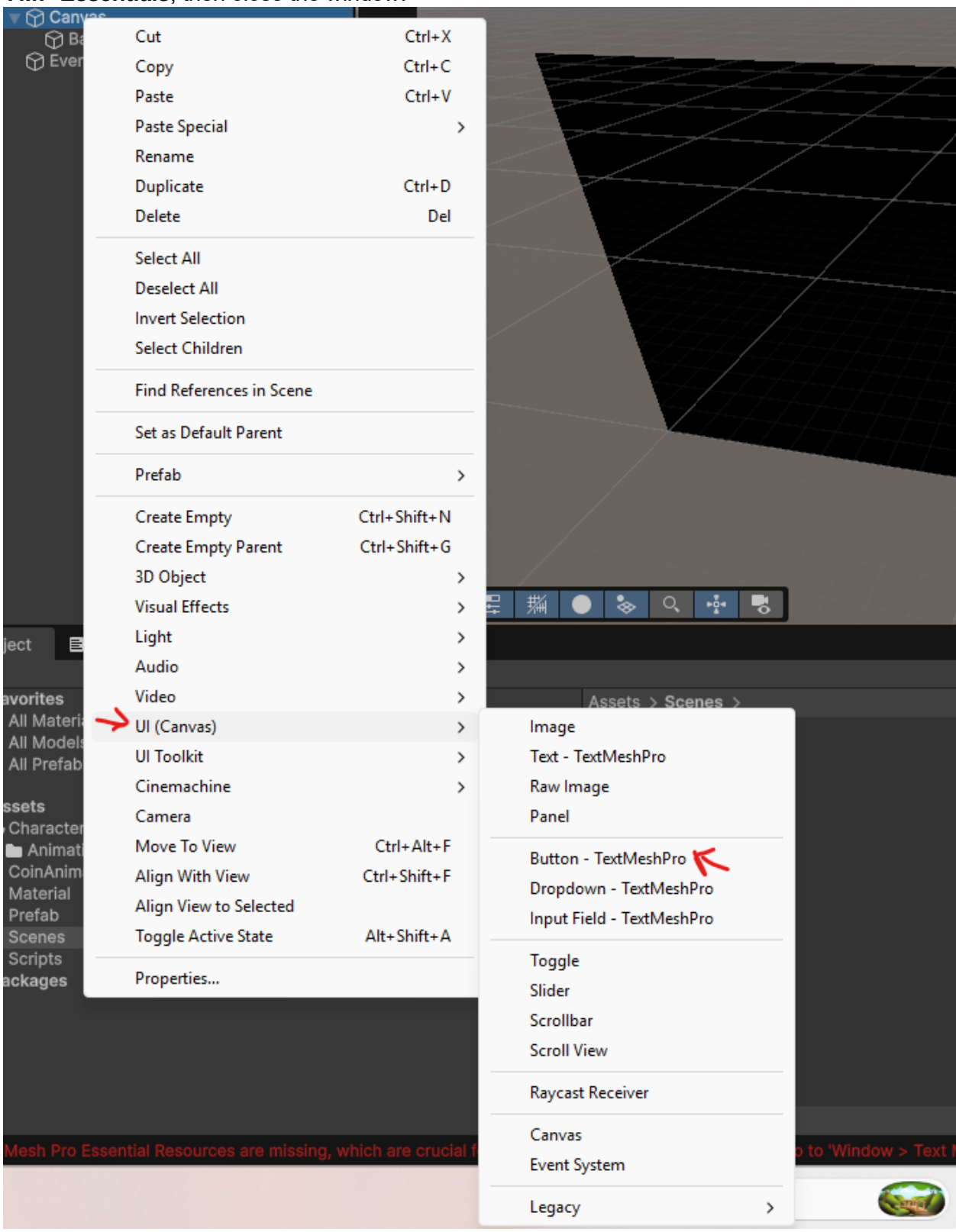
> **Why Scale With Screen Size?** With the default **Constant Pixel Size**, a button that looks fine at 1920 x 1080 looks tiny on a 4K screen. Scale With Screen Size designs the UI for one reference resolution and scales it up or down for every other screen.

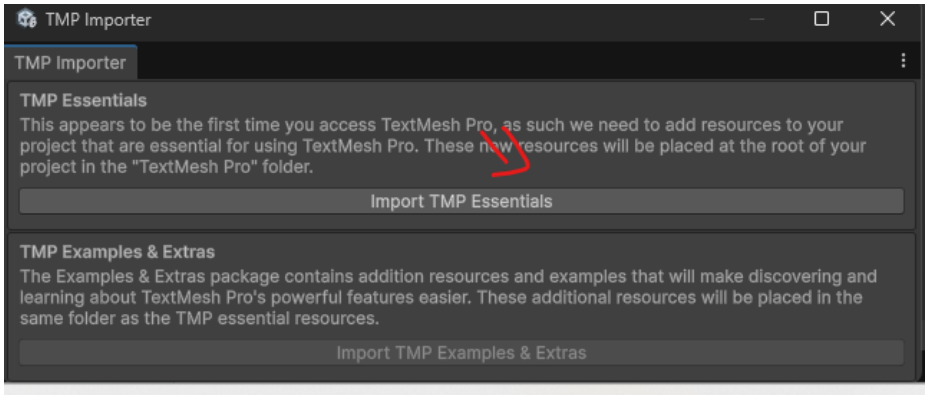
3. **Add a background and a title:**
    - Right-click the **Canvas** > **UI > Image**. Rename it **Background**.
    - In its **Rect Transform**, click the anchor preset square, hold **Alt** and click the bottom-right option (**stretch / stretch**) so it fills the whole screen. Pick a dark **Color**.


Background
Static
Tag Untagged
Layer UI
Rect Transform
center
middle
Pos X
Pos Y
Pos Z
0
0
0
Width
Height
100
100
Anchors
Min
X 0.5
Y 0.5
Max
X 0.5
Y 0.5
Pivot
X 0.5
Y 0.5
Rotation
X 0
Y 0
Z 0
Scale
X 1
Y 1
Z 1
Canvas Renderer
Cull Transparent Mesh
Image
Source Image
None (Sprite)
Color
Material
None (Material)
Raycast Target
Raycast Padding
Maskable
Default UI Material (Material)
Shader
UI/Default
Edit...
Add Component
Background
Activate Windows
Go to Settings to activate Windows.
Background
Image Size: 0x0

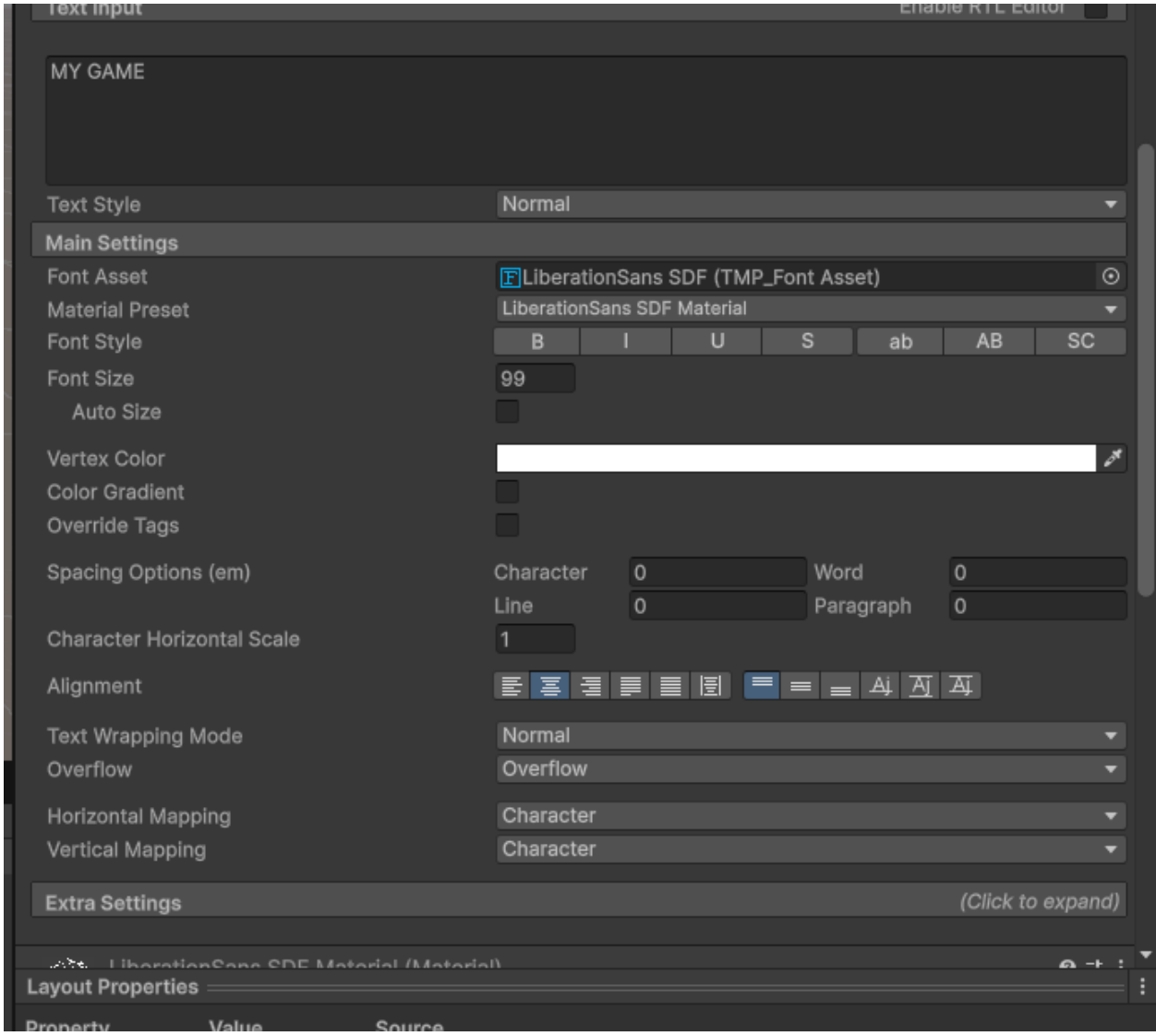

Background
Static
Tag Untagged
Layer UI
Rect Transform
stretch
stretch
Left
Top
Pos Z
0
0
0
Right
Bottom
0
0
Anchors
Min X 0 Y 0
Max X 1 Y 1
Pivot X 0.5 Y 0.5
Rotation X 0 Y 0 Z 0
Scale X 1 Y 1 Z 1
Canvas Renderer
Cull Transparent Mesh
Image
Source Image None (Sprite)
Color
Material None (Material)
Raycast Target
Raycast Padding
Maskable
Default UI Material (Material)
Shader UI/Default
Edit...
Add Component
Background
Activate Windows
Go to Settings to activate Windows.
Background
Image Size: 0x0
ENG
US
7:29 AM
2026-10-07

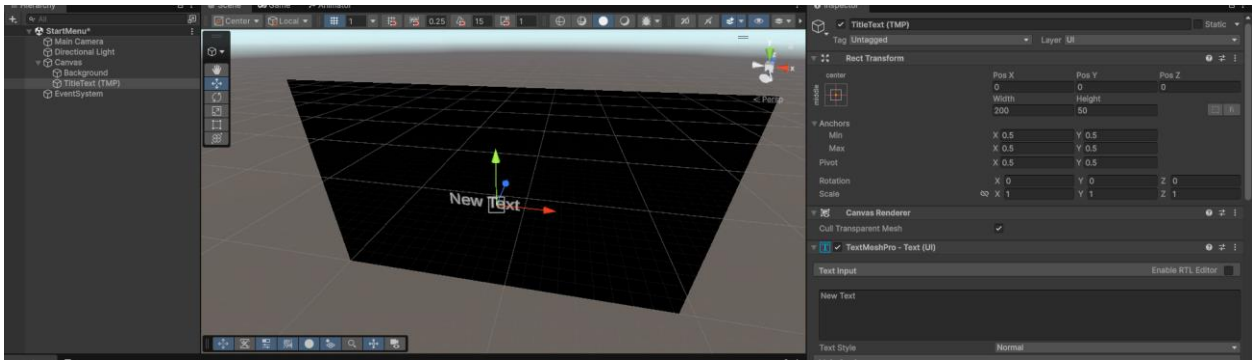
- Right-click the **Canvas** > **UI > Text - TextMeshPro**. If Unity asks, click **Import TMP Essentials**, then close the window.



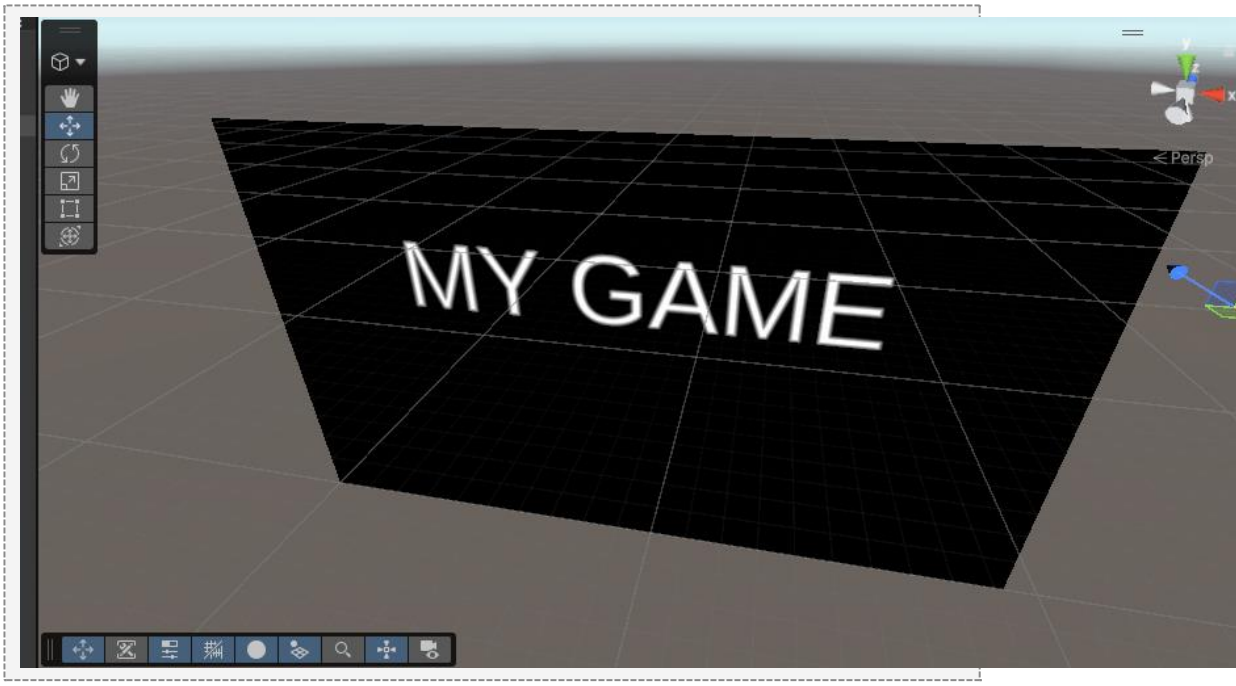
- 
TMP Importer
TMP Importer
TMP Essentials
This appears to be the first time you access TextMesh Pro, as such we need to add resources to your project that are essential for using TextMesh Pro. These new resources will be placed at the root of your project in the "TextMesh Pro" folder.
Import TMP Essentials
TMP Examples & Extras
The Examples & Extras package contains addition resources and examples that will make discovering and learning about TextMesh Pro's powerful features easier. These additional resources will be placed in the same folder as the TMP essential resources.
Import TMP Examples & Extras

- Rename it **TitleText**. Type your game's name in the **Text** box, set **Font Size** to 96, **Alignment** to center and make sure **Auto Size** is off.



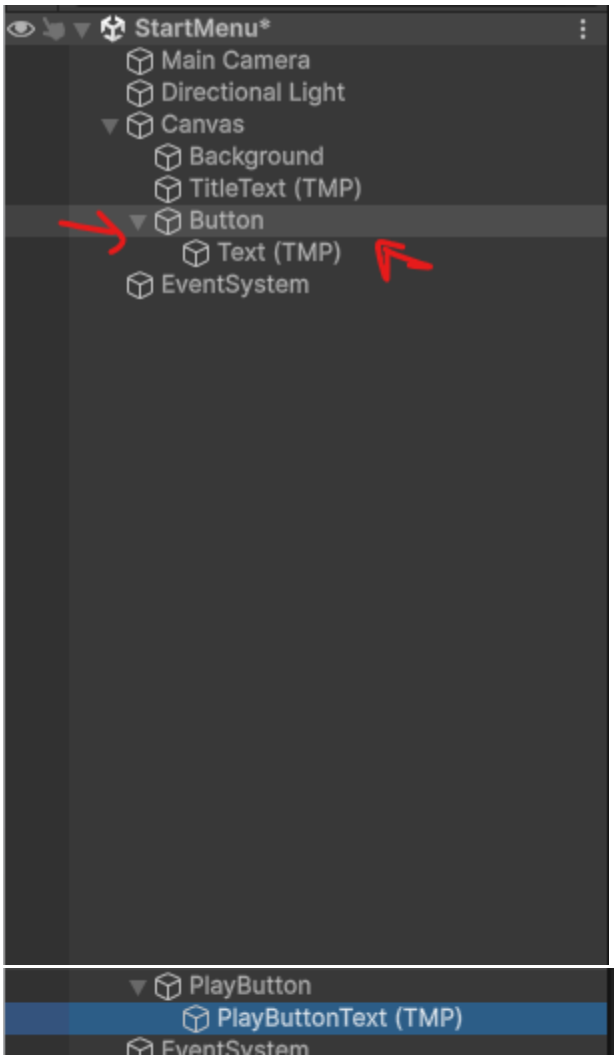


- 

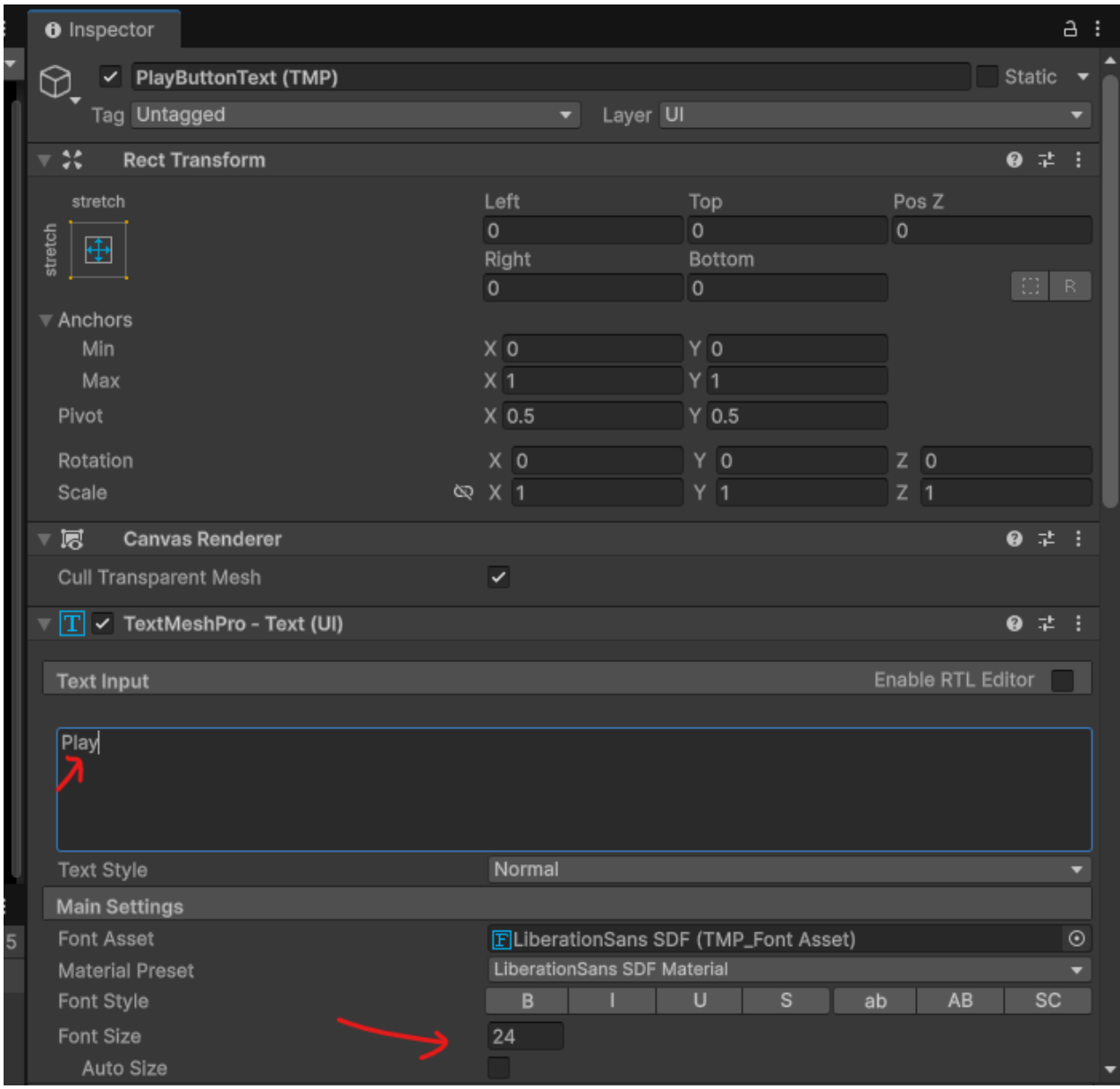
- Anchor preset: hold **Alt + Shift** and click **top-center**. Set **Pos Y** to -200, **Width** 1400, **Height** 160.
- 

> **Anchors in one sentence:** The **anchor** is the point of the screen an element is attached to. A title anchored to the top-center stays 200 pixels below the top of the screen no matter how wide or tall the window becomes.

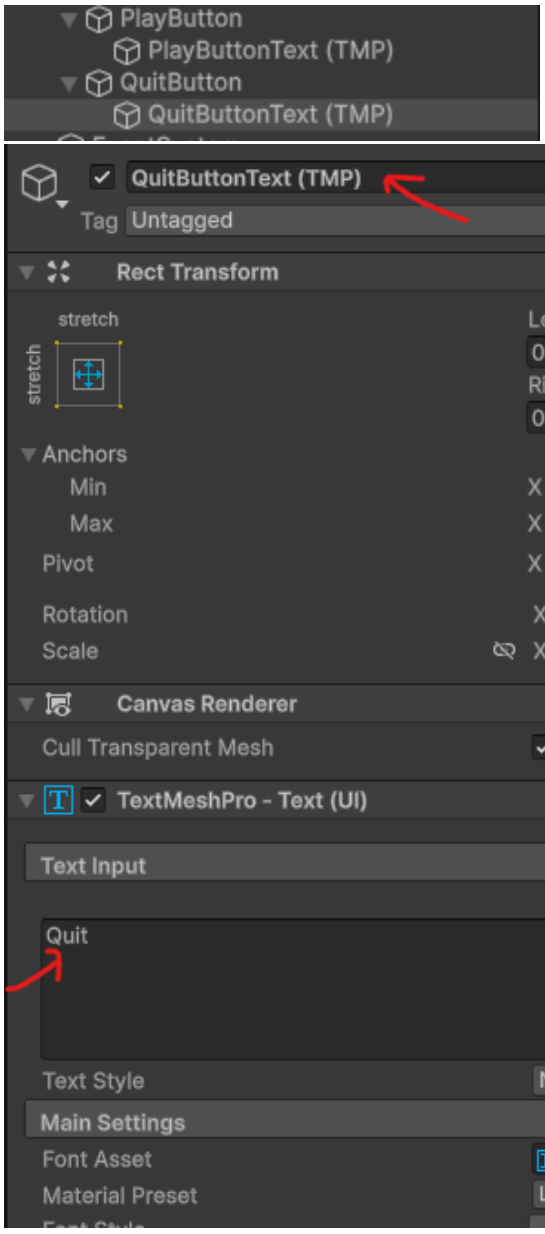
4. **Add the Play and Quit buttons:**
    - Right-click the **Canvas** > **UI > Button - TextMeshPro**. Rename it **PlayButton**. Set **Width** 400, **Height** 100, **Pos X** 0, **Pos Y** 0.

- 

- Expand PlayButton, select its child **Text (TMP)**, change the text to **Play** and the Font Size to 48.



- Select PlayButton and press **Ctrl+D** to duplicate it. Rename the copy **QuitButton**, set **Pos Y** to -150, and change its text to **Quit**.

- 
PlayButton
PlayButtonText (TMP)
QuitButton
QuitButtonText (TMP)
QuitButtonText (TMP)
Static
Tag Untagged
Layer UI
Rect Transform
stretch
stretch
Left
Top
Pos Z
0
0
0
Right
Bottom
0
0
R
Anchors
Min
X 0
Y 0
Max
X 1
Y 1
Pivot
X 0.5
Y 0.5
Rotation
X 0
Y 0
Z 0
Scale
X 1
Y 1
Z 1
Canvas Renderer
Cull Transparent Mesh
TextMeshPro - Text (UI)
Text Input
Enable RTL Editor
Quit
Text Style
Normal
Main Settings
Font Asset
LiberationSans SDF (TMP_Font Asset)
Material Preset
LiberationSans SDF Material


MY GAME
Play
Quit

Task 7: Switch Scenes with a SceneLoader Script

Buttons don't do anything on their own. We will write one small, reusable **SceneLoader** script. The Start Menu uses it now to start the game, and in Task 10 the End scene will use the same script for its **Play Again** and **Main Menu** buttons.

The game scene itself (SampleScene) will **not** have a menu or pause screen. Once the level starts, the only way out is to reach the end of the level.

1. **Create the SceneLoader script:**
    - In the StartMenu scene, right-click the Hierarchy > **Create Empty**, rename it **SceneLoader**.
    - Click **Add Component**, create a new script called **SceneLoader**, and replace its contents with:

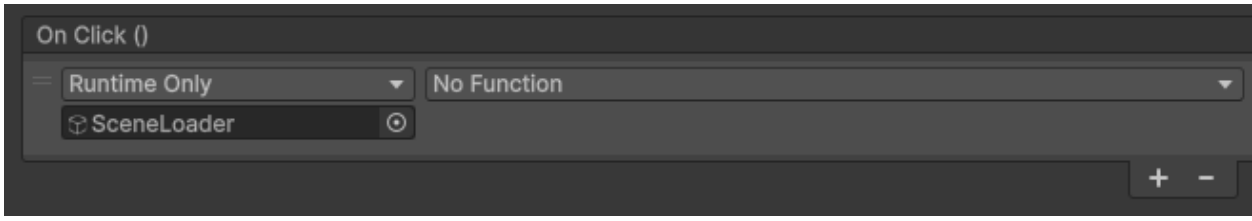
```
using UnityEngine;
using UnityEngine.SceneManagement; // needed for SceneManager

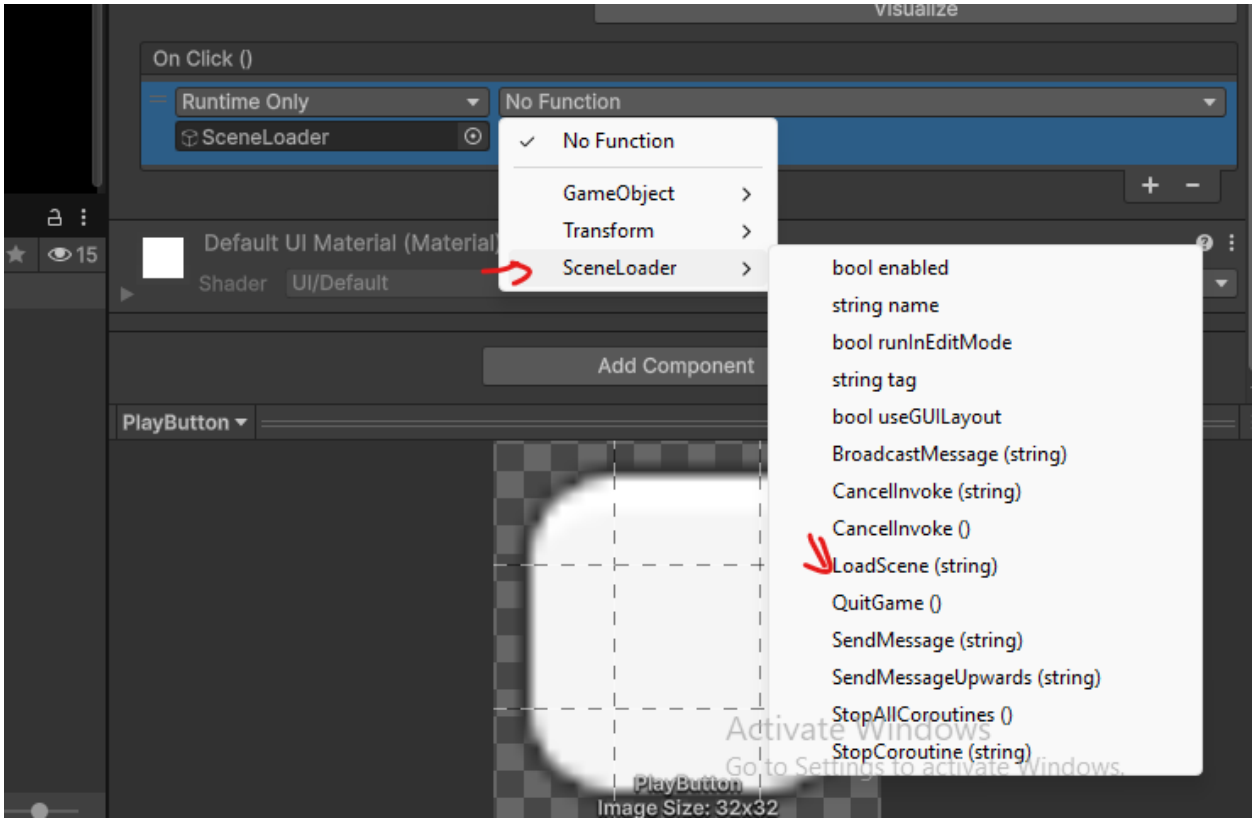
// Loads scenes by name. Buttons call LoadScene() and QuitGame()
// from their On Click list.
public class SceneLoader : MonoBehaviour
{
    // Public so a Button can call it. The scene must be in the
    // Build Profiles scene list or Unity will refuse to load it.
    public void LoadScene(string sceneName)
    {
        SceneManager.LoadScene(sceneName);
    }

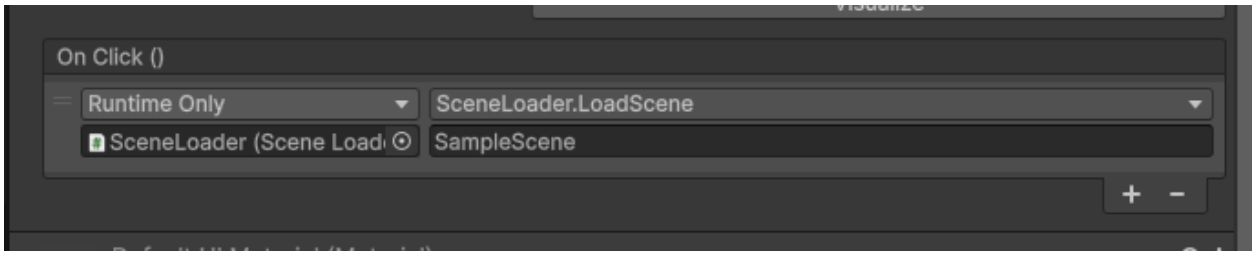
    public void QuitGame()
    {
        Debug.Log("Quit Game");
        // Closes a built game. Does nothing inside the Unity Editor.
        Application.Quit();
    }
}
```

    - `SceneManager.LoadScene()` unloads the current scene (destroying every object in it) and loads the new one.
    - `LoadScene` takes a **string** parameter, so one method works for any scene: the button decides which name to send.
    - `Application.Quit()` closes the game once it is **built**. Inside the Unity Editor it is ignored, which is why we also print **Quit Game** with `Debug.Log`: the Console message proves the button works.

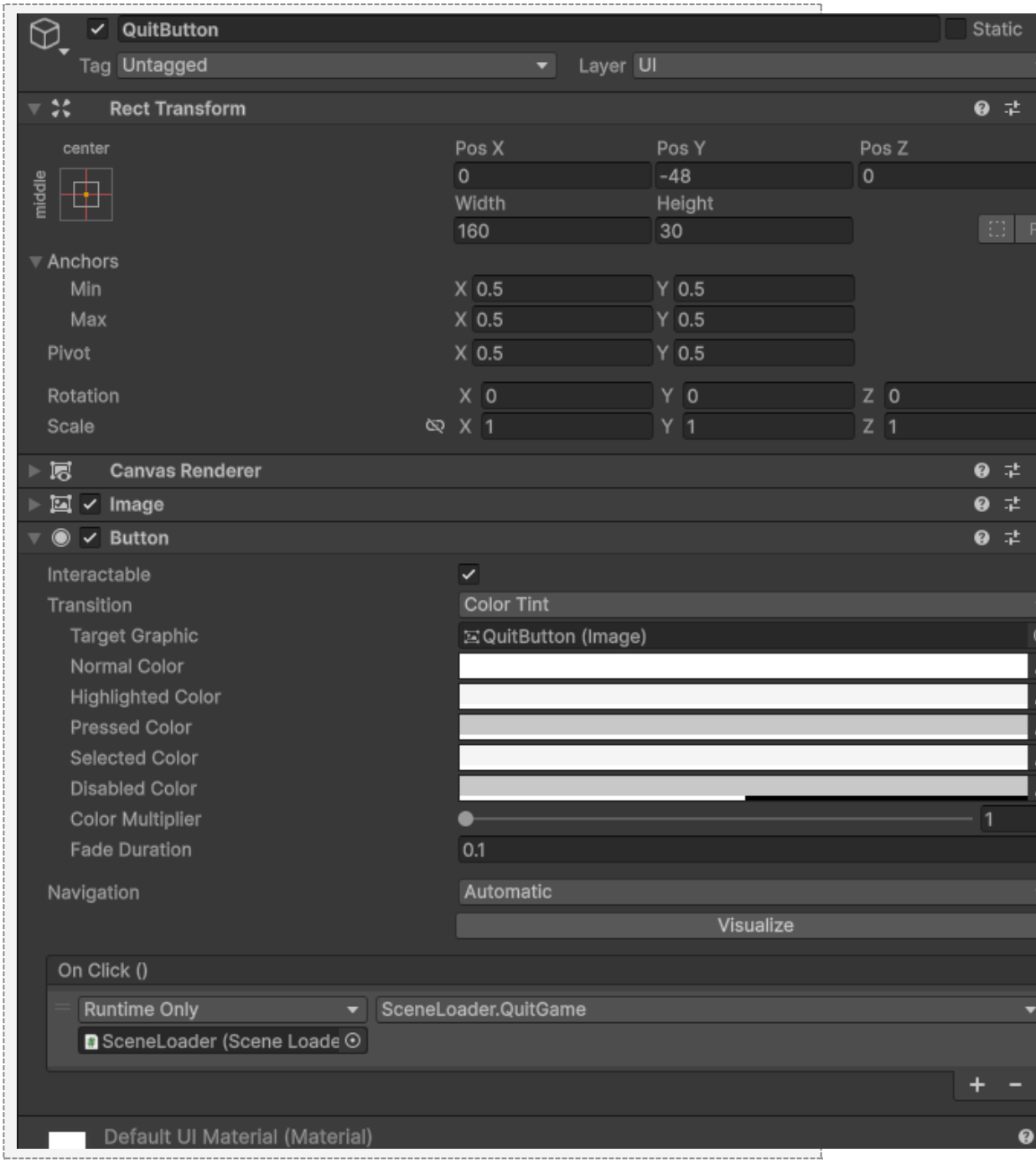
2. **Hook up the buttons:**
    - Select **PlayButton**. At the bottom of the **Button** component, find **On Click ()** and click **+**.

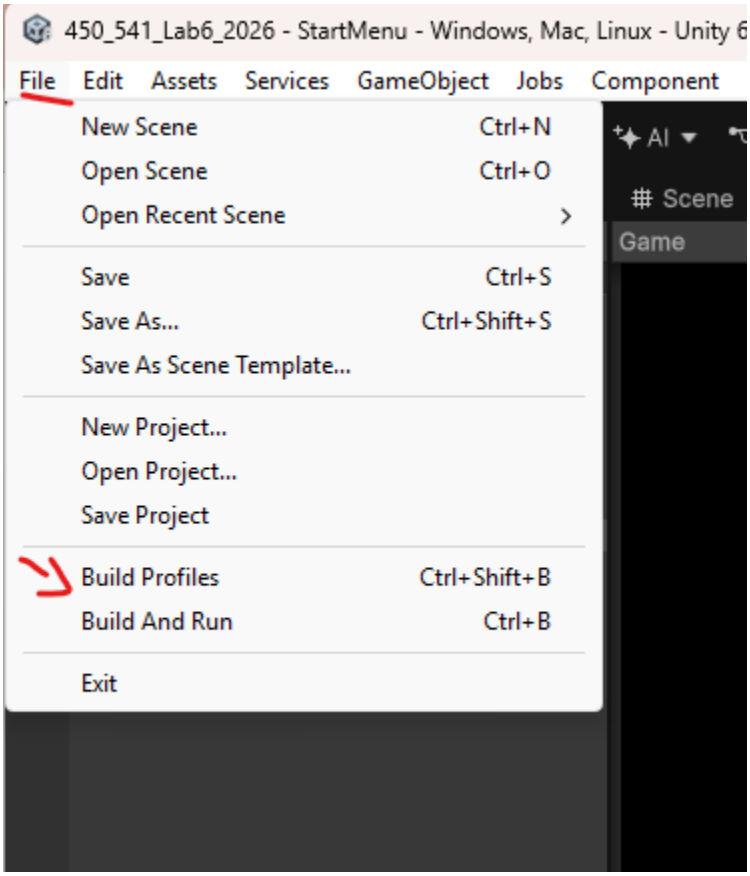
- 

- Drag the **SceneLoader** GameObject from the Hierarchy into the empty object slot.
- In the function dropdown choose **SceneLoader > LoadScene (string)**. A text box appears: type **SampleScene** exactly (it must match the scene file name).
- 

- 

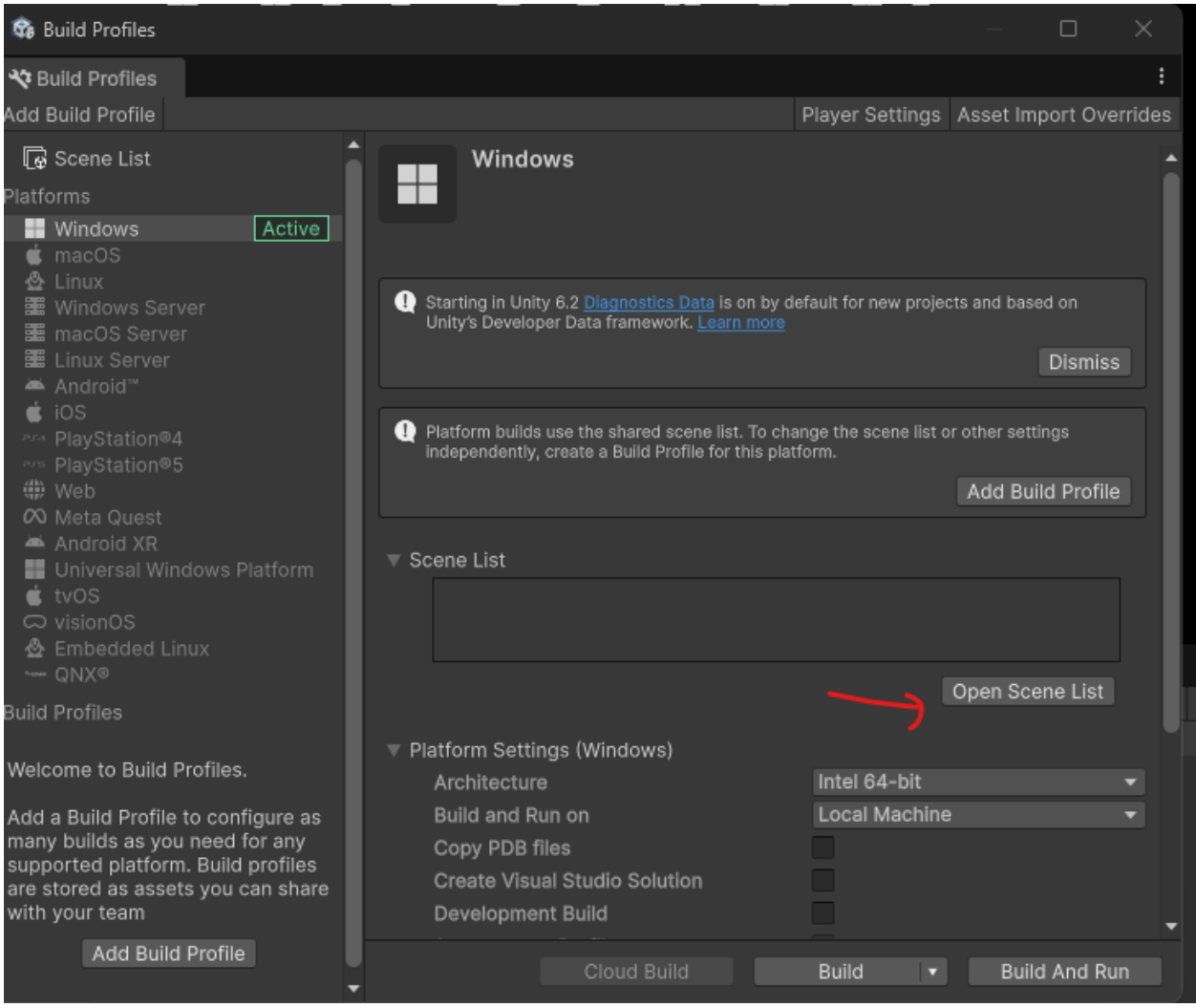
- Select **QuitButton**, add an On Click entry the same way, and choose **SceneLoader > QuitGame ()**.

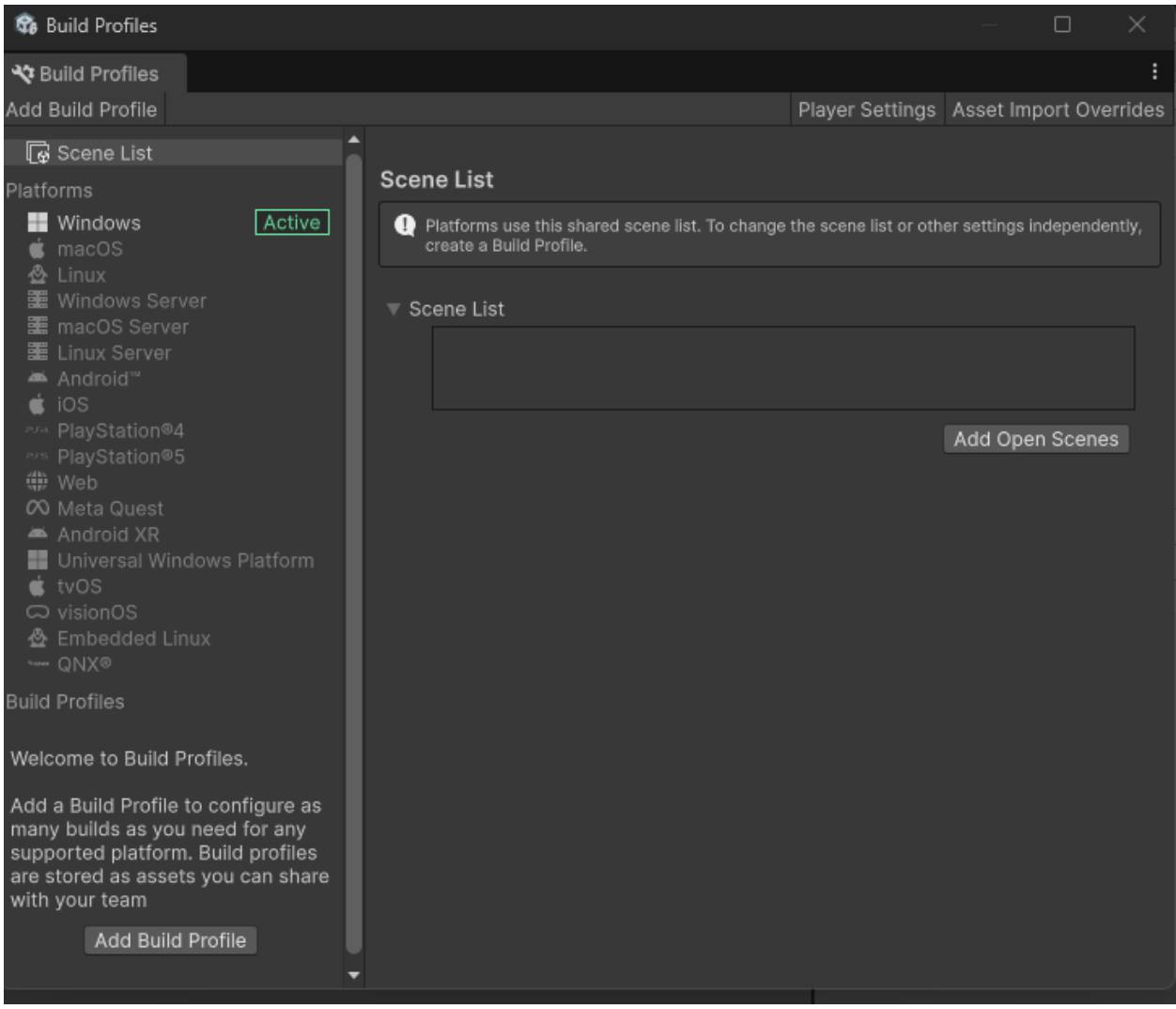

QuitButton
Static
Tag Untagged
Layer UI
Rect Transform
center
middle
Pos X
Pos Y
Pos Z
0
-48
0
Width
Height
160
30
Anchors
Min
X 0.5
Y 0.5
Max
X 0.5
Y 0.5
Pivot
X 0.5
Y 0.5
Rotation
X 0
Y 0
Z 0
Scale
X 1
Y 1
Z 1
Canvas Renderer
Image
Button
Interactable
Transition
Color Tint
Target Graphic
QuitButton (Image)
Normal Color
Highlighted Color
Pressed Color
Selected Color
Disabled Color
Color Multiplier
1
Fade Duration
0.1
Navigation
Automatic
Visualize
On Click ()
Runtime Only
SceneLoader.QuitGame
SceneLoader (Scene Loade
Default UI Material (Material)

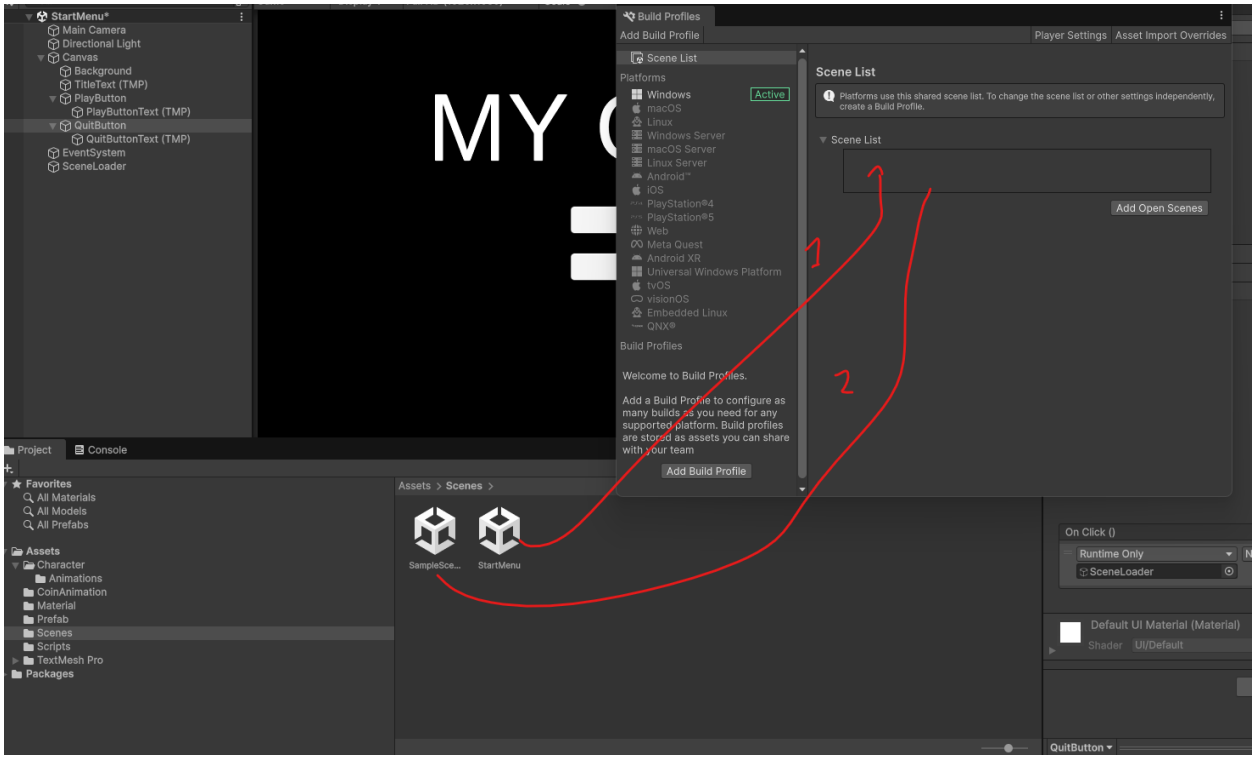
3. **Add the scenes to the Build Profiles scene list:**
    - Open **File > Build Profiles** (Unity 2022 and older: **File > Build Settings**).
    - 

    - In the **Scene List**, make sure both scenes are listed and checked. With StartMenu open, click **Add Open Scenes**, or drag scenes in from the Project

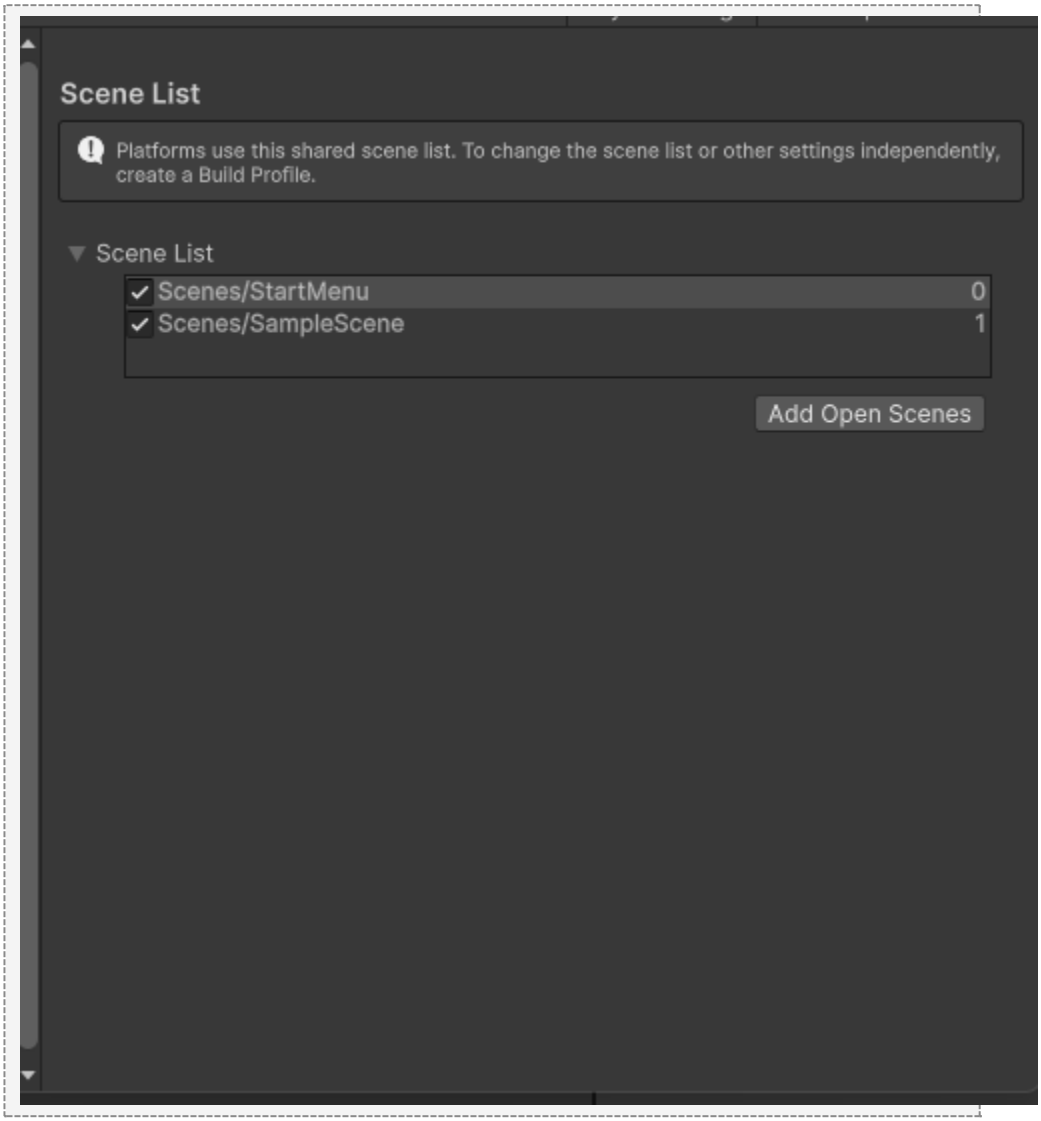
window.


Build Profiles
Build Profiles
Add Build Profile
Player Settings
Asset Import Overrides
Scene List
Platforms
Windows
Active
macOS
Linux
Windows Server
macOS Server
Linux Server
Android™
iOS
PlayStation®4
PlayStation®5
Web
Meta Quest
Android XR
Universal Windows Platform
tvOS
visionOS
Embedded Linux
QNX®
Build Profiles
Welcome to Build Profiles.
Add a Build Profile to configure as many builds as you need for any supported platform. Build profiles are stored as assets you can share with your team
Add Build Profile
Windows
Starting in Unity 6.2 Diagnostics Data is on by default for new projects and based on Unity's Developer Data framework. Learn more
Dismiss
Platform builds use the shared scene list. To change the scene list or other settings independently, create a Build Profile for this platform.
Add Build Profile
Scene List
Open Scene List
Platform Settings (Windows)
Architecture
Intel 64-bit
Build and Run on
Local Machine
Copy PDB files
Create Visual Studio Solution
Development Build
Cloud Build
Build
Build And Run

- 
Build Profiles
Build Profiles
Add Build Profile
Player Settings
Asset Import Overrides
Scene List
Platforms
Windows
Active
macOS
Linux
Windows Server
macOS Server
Linux Server
Android™
iOS
PlayStation®4
PlayStation®5
Web
Meta Quest
Android XR
Universal Windows Platform
tvOS
visionOS
Embedded Linux
QNX®
Build Profiles
Welcome to Build Profiles.
Add a Build Profile to configure as many builds as you need for any supported platform. Build profiles are stored as assets you can share with your team
Add Build Profile
Scene List
Platforms use this shared scene list. To change the scene list or other settings independently, create a Build Profile.
Scene List
Add Open Scenes

- 

- Drag **StartMenu** to the top so it is index **0**, and **SampleScene** is index **1**. Scene 0 is the one a built game opens first. (You will add the End scene as index 2 in Task 10.)



> **Error "couldn't be loaded because it has not been added to the build settings"?** The scene isn't in the Scene List, or the name typed in the button doesn't match the file name exactly (capital letters count).

4. **Test the scene flow:**
    - Open **StartMenu** and press **Play**. Click **Play**: the level should load. Stop, press Play again, open the **Console** and click **Quit**: the Console should show **Quit Game**. (The game only really closes in a build.)

> **Level looks darker after loading it from the menu?** This only happens in the Editor. Open SampleScene, go to **Window > Rendering > Lighting** and click **Generate Lighting**. Unity then saves the scene's lighting data so it looks the same however it is loaded.

5. **Check your understanding:**
    - What happens to every GameObject in the StartMenu scene (the Canvas, the buttons, the SceneLoader) when `SceneManager.LoadScene("SampleScene")` runs?

    Every GameObject in the StartMenu is destroyed.

    - Why is it better to give `LoadScene` a string parameter than to write separate methods like `LoadGame()` and `LoadMenu()`?

    To keep the code reusable.

Task 8: Add a HUD

A **HUD** (heads-up display) is the UI drawn on top of the game while you play. A good HUD stays out of the way. Ours shows only two things in the top-left corner: the score from Lab 4, and a small **double jump icon with the time remaining** that only appears while the double jump power-up is active (you will create the power-up in Task 9).



Figure 3: HUD layout.

1. **Create the HUD Canvas:**
    - Open **SampleScene**. Right-click the Hierarchy > **UI > Canvas** and rename it **HUD**.
    - Set its **Canvas Scaler** exactly like the menu: **Scale With Screen Size**, **1920 x 1080**, **Match 0.5**.
2. **Add the score text:**
    - Right-click **HUD** > **UI > Text - TextMeshPro**, rename it **ScoreText**.
    - Anchor preset: hold **Alt + Shift** and click **top-left**. Set **Pos X** 40, **Pos Y** -40, **Width** 500, **Height** 80.
    - Set the text to **Score: 0**, Font Size 56, **Bold**.

3. **Create the HUDController script:**
    - Select **HUD**, click **Add Component**, create a script called **HUDController** and replace its contents with:

```
using UnityEngine;
using TMPro; // TMP_Text

// Updates the HUD every frame from the ScoreManager and PlayerPowerUps.
public class HUDController : MonoBehaviour
{
    [Header("Score")]
    [SerializeField] private TMP_Text scoreText;

    private void Update()
    {
        UpdateScore();
    }

    private void UpdateScore()
    {
        // The Score property from Lab 4: anyone can read it
        scoreText.text = "Score: " + ScoreManager.Instance.Score;
    }

}
```

    - In Lab 4, ScoreManager's comment said `Score` was public "for a UI later". This is that UI. Thanks to the singleton, the HUD can read `ScoreManager.Instance.Score` without any reference in the Inspector.

Task 9: Add an End Scene that Shows the Score

Every game needs an ending. We will put a **LevelEnd** zone at the end of the level. When the player walks into it, the game switches to an **EndScene** that shows the final score, with buttons to play again or return to the Start Menu.

There is one problem to solve first. When SampleScene unloads, the **ScoreManager** is destroyed along with its **Score**, so the End scene can't ask it for the score. The fix is a **static** variable.

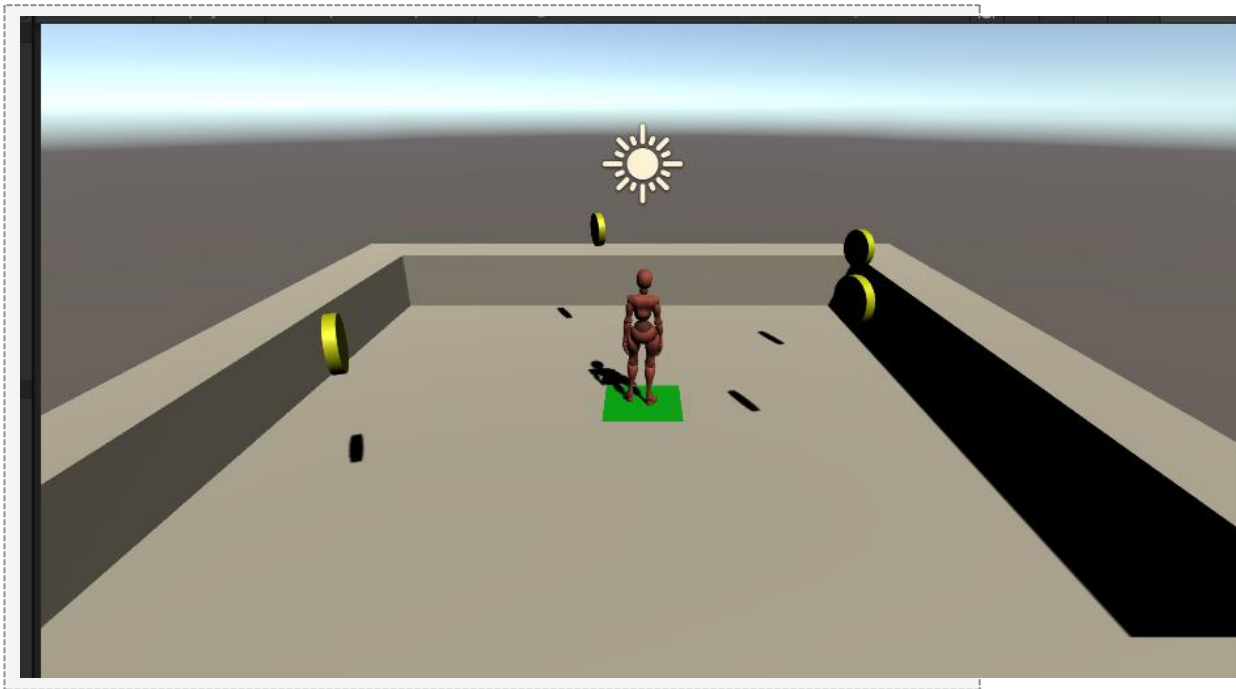
1. **Save the final score in ScoreManager:**
    - Open **ScoreManager**. Add this property and method inside the class (for example just below `AddScore`):

```
// "static" means this value belongs to the ScoreManager CLASS,
// not to one ScoreManager object. Objects are destroyed when a
// new scene loads, but static values stay in memory.
public static int FinalScore { get; private set; }

// Called just before we leave the level
public void SaveFinalScore()
{
    FinalScore = Score;
}
```

> **Static vs. instance:** `Score` is stored **inside** the ScoreManager object, so it disappears with that object. `FinalScore` is stored **on the class itself**, so any scene can read it with `ScoreManager.FinalScore`, even a scene that has no ScoreManager in it. You already used this idea for `Instance` in Lab 4.

2. **Build the LevelEnd zone:**
    - In **SampleScene**, right-click the Hierarchy > **Create Empty**, rename it **LevelEnd**, and place it at the far end of your level.
    - Right-click **LevelEnd** > **3D Object > Cylinder**, rename it **Visuals**, set **Scale** to 2, 0.05, 2 and give it a green material so it looks like a finish pad. Remove its **Capsule Collider**.
    - Select **LevelEnd** (the parent), add a **Box Collider**, check **Is Trigger**, and set **Size** to 2, 2, 2 and **Center Y** to 1.



3. **Create the LevelEndTrigger script:**
    - Select **LevelEnd**, click **Add Component**, create a script called **LevelEndTrigger** and replace its contents with:

```
using UnityEngine;
using UnityEngine.SceneManagement;

// Put on a trigger at the end of the level.
// When the player enters it, save the score and load the End scene.
public class LevelEndTrigger : MonoBehaviour
{
    [SerializeField] private string endSceneName = "EndScene";

    private void OnTriggerEnter(Collider other)
    {
        // Only the player can finish the level
        if (!other.CompareTag("Player")) return;

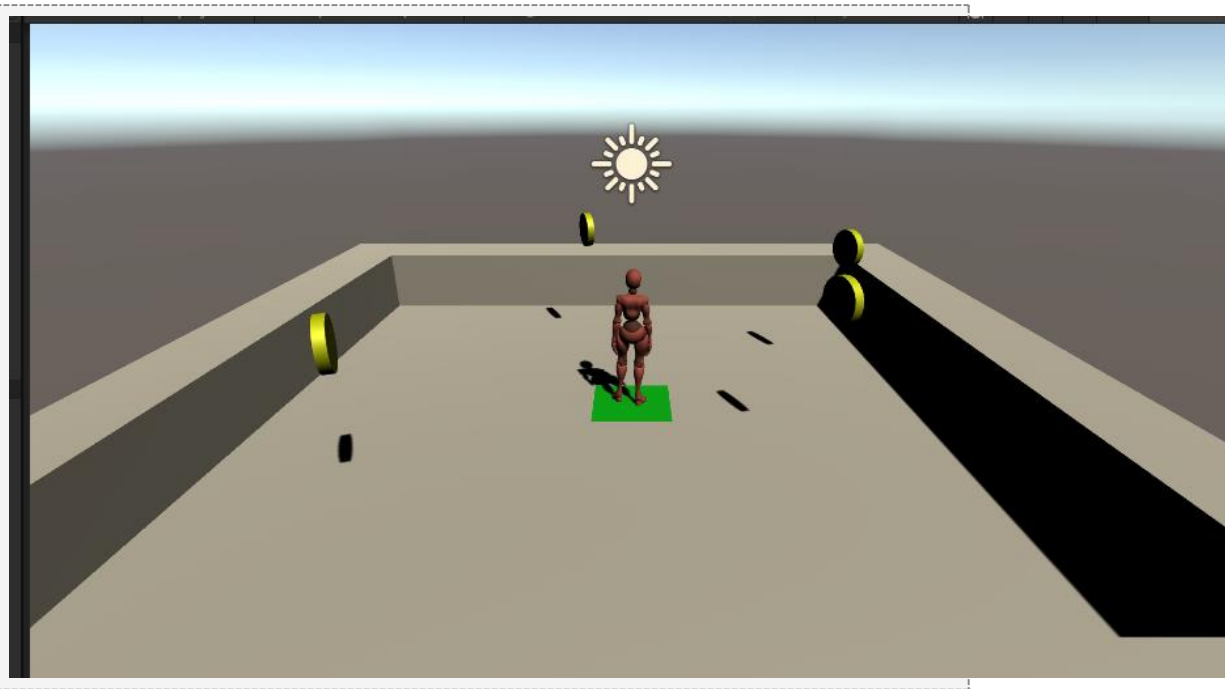
        // Save the score BEFORE loading: the ScoreManager is about
        // to be destroyed together with the rest of SampleScene
        ScoreManager.Instance.SaveFinalScore();

        SceneManager.LoadScene(endSceneName);
    }
}
```

4. **Create the EndScene:**
    - Save SampleScene. In the **Scenes** folder create a new scene called **EndScene** and open it.

- Build its UI the same way as the Start Menu (Task 6): a **Canvas** with **Scale With Screen Size** (1920 x 1080, Match 0.5) and a full-screen **Background** Image.
- Add a **UI > Text - TextMeshPro** called **TitleText** anchored **top-center**, Pos Y -200, with the text **Level Complete!** and Font Size 96.
- Add another TextMeshPro called **FinalScoreText** anchored **middle-center**, Pos Y 80, Width 1000, Height 120, Font Size 72, centered. Type **Final Score: 0** as placeholder text.
- Add two **Button - TextMeshPro** objects: **PlayAgainButton** (Pos Y -100, text **Play Again**) and **MainMenuButton** (Pos Y -250, text **Main Menu**), each 400 x 100.



- 

5. **Show the score with an EndScreen script:**
    - Create an empty GameObject called **EndScreen**, add a new script called **EndScreen** and replace its contents with:

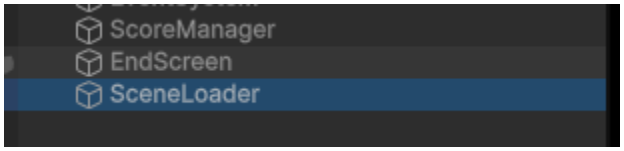
```
using UnityEngine;
using TMPro;

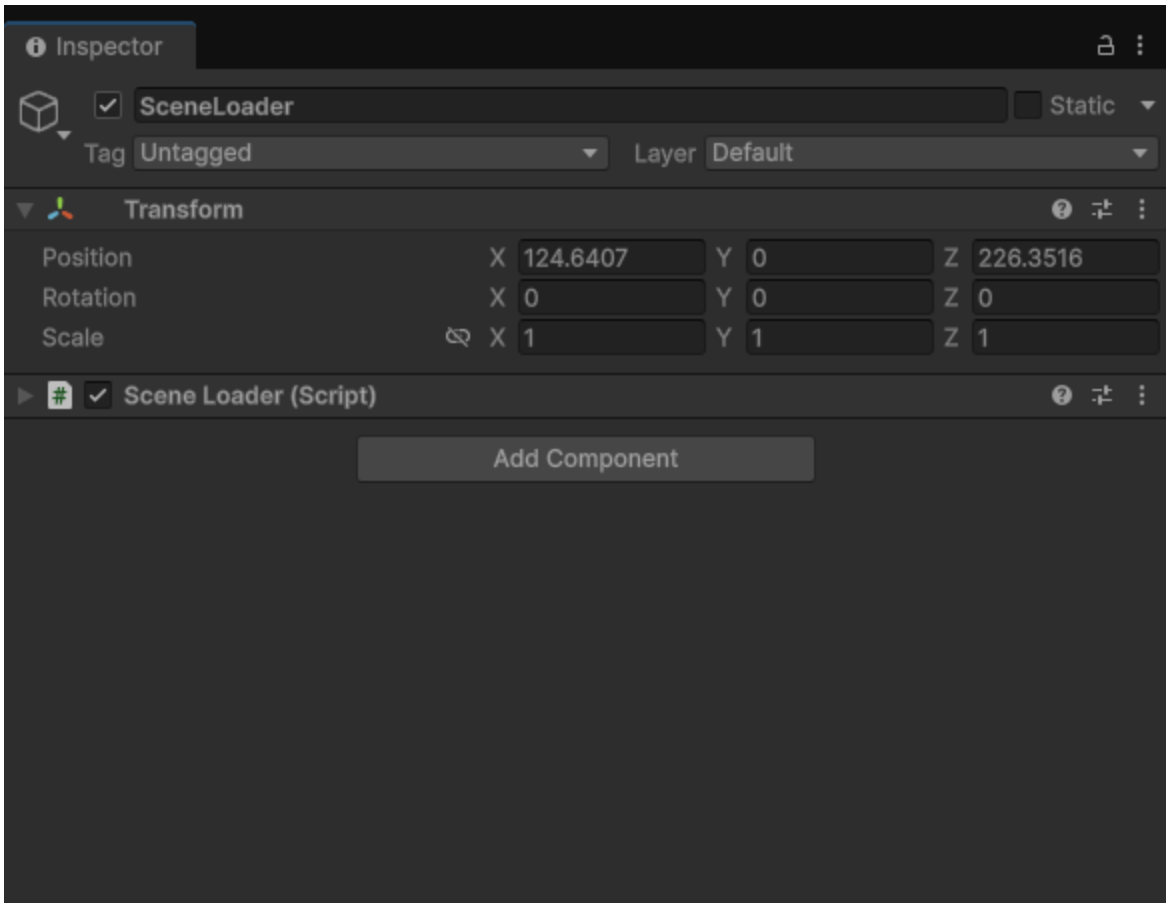
// Shows the score the player finished the level with.
public class EndScreen : MonoBehaviour
{
    [SerializeField] private TMP_Text finalScoreText;

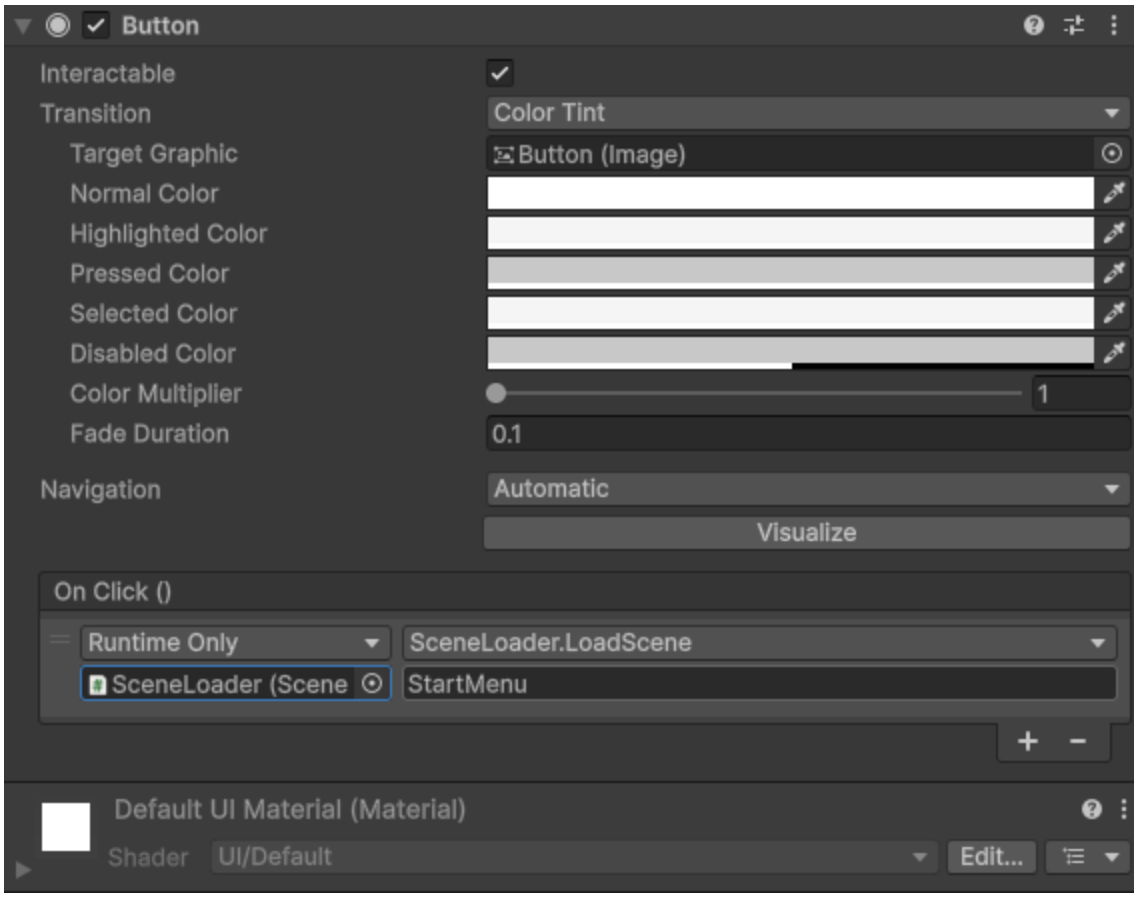
    private void Start()
    {
        // No ScoreManager object exists in this scene, but we can
        // still read the static FinalScore through the class name
        finalScoreText.text = "Final Score: " + ScoreManager.FinalScore;
    }
}
```

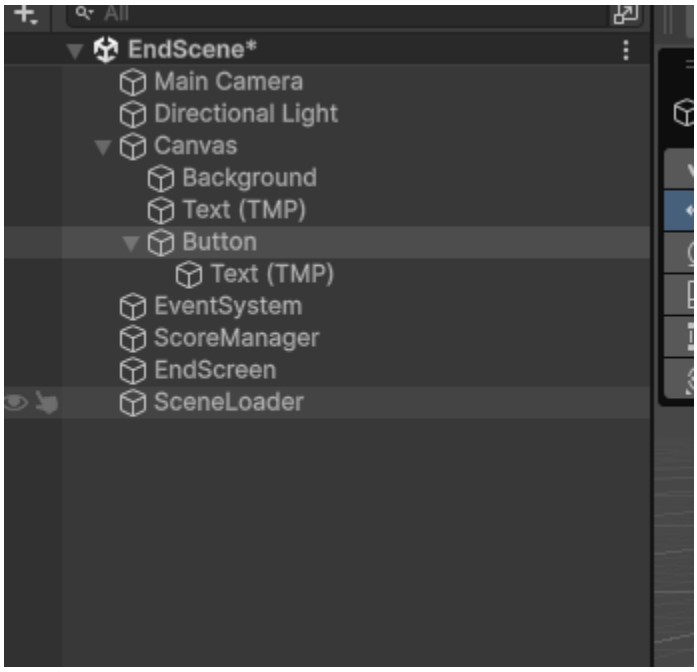
    - Drag **FinalScoreText** into the **Final Score Text** slot.

6. **Hook up the buttons:**
    - Create an empty GameObject called **SceneLoader** and add your **SceneLoader** script from Task 7. Reusing it is exactly why we gave `LoadScene` a string parameter.

    - **PlayAgainButton** > On Click () > **SceneLoader > LoadScene (string)** > type **SampleScene**.
    - **MainMenuButton** > On Click () > **SceneLoader > LoadScene (string)** > type **StartMenu**.
    - 

- 
Inspector
SceneLoader
Static
Tag
Untagged
Layer
Default
Transform
Position
X 124.6407
Y 0
Z 226.3516
Rotation
X 0
Y 0
Z 0
Scale
X 1
Y 1
Z 1
Scene Loader (Script)
Add Component

- 

- 

7. **Add EndScene to the Build Profiles scene list:**
    - Open **File > Build Profiles** and add **EndScene** so the list reads **0 StartMenu**, **1 SampleScene**, **2 EndScene**.



8. **Test the full game loop:**
   - Open **StartMenu** and press **Play**. Click **Play**, collect a few coins, then walk onto the **LevelEnd** pad.
   - The End scene should show **Final Score** with the number from your HUD.
   - Click **Play Again**: the level reloads with the score back at **0**. Finish again, then click **Main Menu** to return to the Start Menu.

> **Final Score always shows 0?** Check that `SaveFinalScore()` is called **before** `SceneManager.LoadScene` in LevelEndTrigger, and that the Player's tag is **Player**.
>
> **Nothing happens on the pad?** The Box Collider on **LevelEnd** must have **Is Trigger** ticked, and it must be big enough for the player's capsule to enter.

9. **Check your understanding:**
   - Why can the End scene read `ScoreManager.FinalScore` but not `ScoreManager.Instance.Score`?

Because FinalScore is a static variable and not Instance.Score

---

   - If you open EndScene directly in the Editor and press Play, what score does it show? Why?

It shows a score of 0 because the game hasn't been played, and the default value we set is 0.

Final Steps:

1. **Test Your Project:**
   - Open **StartMenu** and press **Play**. Click **Play** to load the level.
   - Check that everything from the starter project still works: walking and running, jump, fall, roll and coins.
   - The HUD shows the score.
   - Walk onto the **LevelEnd** pad: the End scene shows your final score. **Play Again** restarts the level and **Main Menu** returns to the Start Menu, where **Quit** prints **Quit Game** in the Console (and closes the game in a build).

- Keep the Animator window open on the Player while testing: watch Falling and Flip light up live.

2. **Submission:**
   - Save your fork of the repo.
   - Submit your Assets folder zipped with this document filled in and saved in the assets folder.

https://github.com/J0YSICK/450_541_Lab6_2026.git